OnBuch+

SIGNED COURSE • PREMIUM

COMPUTER SCIENCE • UPPER SIXTH | UPPER SIXTH — COMPUTER SCIENCE

CHAPTER 9

EXPLORING SYSTEM SOFTWARE CONCEPTS.

SUGGESTED TIME: 6 PERIODS

IN THIS CHAPTER

This chapter explores system software: its role as the bridge between hardware and application software, the place of utility programs, and the operating system — its types, evolution, interfaces, installation and file storage management. Students gain the theoretical foundation and practical reflexes needed to install, use and maintain a computer system, in line with the GCE Advanced Level requirements.

- By the end of this chapter I can define system software and distinguish it from application software with concrete examples.
- By the end of this chapter I can describe the main categories of system software (operating systems, translators, loaders/linkers, device drivers, utilities).
- By the end of this chapter I can explain the role and give examples of common utility software (antivirus, compression, backup, disk tools).
- By the end of this chapter I can trace the evolution of operating systems and classify OS types (batch, time-sharing, real-time, network, distributed, mobile, embedded).
- By the end of this chapter I can compare command-line and graphical user interfaces and use basic CLI commands.
- By the end of this chapter I can describe the steps of installing an operating system, including partitioning and formatting.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE — complete, illustrated, step by step

METHODS — and worked examples

EXERCISES — exam style + solutions

SUMMARY SHEET — the essentials to remember

CONTENTS

1. Introduction to System Software
2. System and Utility Software
3. Types and Evolution of Operating Systems
4. Operating System Interfaces
5. OS Installation and File Storage
6. Integration activity
7. Methods and worked examples
8. Exercises
9. Solutions to the exercises
10. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can define system software and distinguish it from application software with concrete examples.
- By the end of this chapter I can describe the main categories of system software (operating systems, translators, loaders/linkers, device drivers, utilities).
- By the end of this chapter I can explain the role and give examples of common utility software (antivirus, compression, backup, disk tools).
- By the end of this chapter I can trace the evolution of operating systems and classify OS types (batch, time-sharing, real-time, network, distributed, mobile, embedded).
- By the end of this chapter I can compare command-line and graphical user interfaces and use basic CLI commands.
- By the end of this chapter I can describe the steps of installing an operating system, including partitioning and formatting.
- By the end of this chapter I can explain file storage concepts: files, folders, file systems, paths and file management operations.
- By the end of this chapter I can carry out a supervised OS installation or maintenance task and justify each step.

PREREQUISITES

- Hardware components of a computer system (CPU, memory, storage devices) — Lower Sixth

- Basic distinction between hardware and software — Lower Sixth
- Files, folders and basic file manipulation in a GUI — Lower Sixth / everyday practice
- Number systems and units of storage (bit, byte, KB, MB, GB) — Lower Sixth

1 INTRODUCTION TO SYSTEM SOFTWARE

At a cybercafé in Buea, a student inserts a flash drive into a computer that refuses to start: the screen stays black and displays the cold message "No bootable device found". The owner explains that the machine "lost its Windows" after a power cut during the rainy season. Meanwhile, the neighbouring machine is so slow that customers complain, yet a simple disk cleanup and uninstalling useless programs could fix it. Who decides how the computer starts, manages memory, talks to the printer, and organises files? The answer is **system software** — the invisible manager without which no application, not even WhatsApp Desktop or Microsoft Word, could ever run.

In this section we answer three fundamental questions: What is software? What distinguishes system software from application software? And why can no application run without system software underneath it?

1.1 Software and Its Two Great Families

DEFINITION **Software**

Software is the set of programs, procedures and data that tell a computer's hardware what to do and how to do it. Unlike hardware, which is physical and tangible, software is intangible: it exists as instructions stored in memory and executed by the processor.

A computer without software is like a generator without fuel: the machine is complete, but it does nothing useful. Every task a computer performs — booting up, displaying a photo, printing an invoice, playing music — is driven by some program.

All software can be divided into **two great families**:

- **System software**: programs that control, coordinate and manage the computer's hardware resources and provide a platform on which other programs run. Examples: Windows 11, Ubuntu Linux, Android, device drivers, antivirus tools.
- **Application software**: programs that perform tasks directly useful to the user. Examples: Microsoft Word, Excel, WhatsApp, a school fees management program, a football game.

DEFINITION **System software**

System software is the collection of programs that control, coordinate and manage the hardware resources of a computer system (processor, memory, storage, input/output devices) and provide a platform — a working environment — on which application software can be installed and executed. The user rarely interacts with it directly; it works in the background.

DEFINITION **Application software**

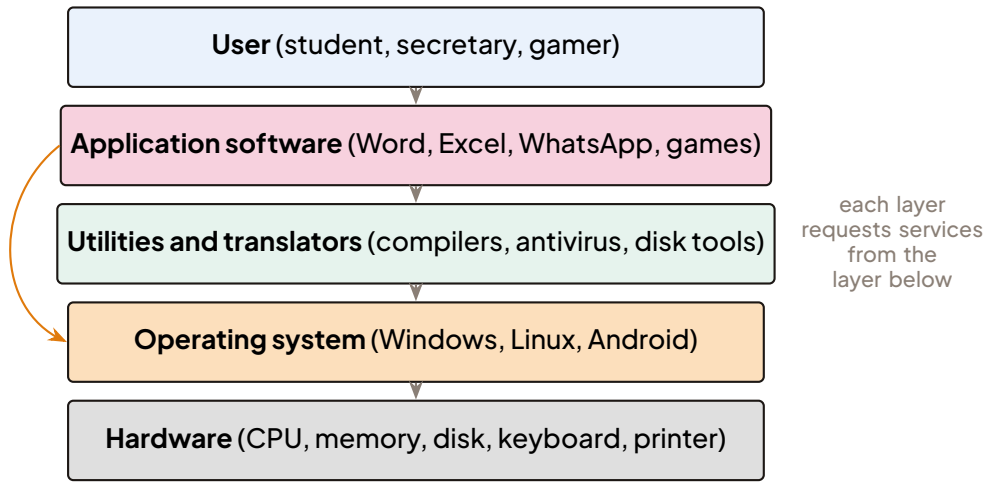
Application software is a program (or a set of programs) designed to help the end user accomplish a specific task: writing a letter, calculating a budget, browsing the Internet, editing a video. Application software always runs on top of system software and requests services (memory, files, screen, network) from it.

WARNING **The key contrast**

System software serves the machine; application software serves the user. When the cybercafé computer in Buea "lost its Windows", every application on it — Word, the browser, the games — became useless, even though their files were still on the disk. Remove the application and the machine still works; remove the system software and nothing works.

1.2 The Layered Model of a Computer System

A modern computer is organised like the floors of a building: each layer rests on the one below and offers services to the one above. This is called the **layered model** (or onion model) of a computer system.



The layered model of a computer system: hardware at the bottom, the user at the top, with the operating system as the essential bridge.

Reading the diagram from bottom to top:

1. **Hardware**: the physical components — CPU, RAM, hard disk, keyboard, screen, printer.
2. **Operating system**: the master program that drives the hardware and shares it fairly among

programs.

3. **Utilities and language translators**: support programs (compilers, antivirus, disk cleanup tools) that help maintain the system or produce other programs.
4. **Application software**: the programs the user actually wants to use.
5. **User**: the person at the top, who normally touches only the application layer.

KEY POINTS **Why layering matters**

An application never talks to the hardware directly. When Microsoft Word prints a document, it asks the operating system; the operating system asks the printer driver; the driver sends signals to the printer. This is why the same version of Word can print on hundreds of different printer models: the OS hides the hardware differences.

1.3 Main Categories of System Software

System software is not a single program but a family. The main categories are:

1. **Operating systems (OS)**: the core of system software. They boot the machine, manage all resources and provide the user interface. Examples: Microsoft Windows, Linux (Ubuntu, Debian), macOS, Android, iOS.
2. **Language translators**: programs that convert source code written by programmers into machine code executable by the CPU.
   - Assembler: translates assembly language (e.g. `MOV AX, 5`) into machine code.
   - Compiler: translates a whole high-level program (C, Pascal) into an executable file at once.
   - Interpreter: translates and executes a high-level program line by line (Python, classic BASIC).
3. **Loaders and linkers**: the linker combines separately compiled modules and libraries into one executable program; the loader copies that program from disk into main memory and starts its execution.
4. **Device drivers**: small programs, one per device type, that allow the OS to communicate with a specific piece of hardware (printer driver, graphics driver, flash-drive driver). Without the correct driver, a device is invisible to the system.
5. **Utility software**: maintenance and housekeeping programs that keep the system healthy: antivirus tools, disk cleanup, disk defragmenters, backup tools, file compression tools (WinRAR), partition managers. The disk cleanup that would have saved the slow machine in our Buea cybercafé is a utility.

1.4 The Operating System as Resource Manager

The most important piece of system software is the operating system. Its central role is to act

as a **resource manager**: it shares the computer's limited resources among all running programs, fairly and safely.

Resource	What the OS does with it
CPU (processor)	Decides which program uses the processor at each instant and for how long (scheduling), so that Word, the browser and the music player all appear to run at the same time.
Main memory (RAM)	Allocates memory space to each program, protects one program's data from another, and frees memory when a program closes.
Storage (disks)	Organises data into files and folders, keeps track of free and used space, and controls who may read or write each file.
Input/output devices	Receives data from keyboard and mouse, sends output to screen and printer, through the appropriate device drivers.

EXEMPLE **The OS at work in one click**

You double-click a photo on your desktop. In a fraction of a second, the OS: (1) detects the mouse click (input management); (2) finds the photo file on the disk (storage management); (3) loads an image-viewer program into RAM (memory management); (4) gives it CPU time to decode the image (processor management); (5) displays the result on the screen through the graphics driver (output management). One click, five services — all invisible.

1.5 Why Applications Cannot Run Without System Software

PROPERTY **Dependence of applications on system software**

Every application program depends on system software to run, because an application contains no instructions for driving hardware directly: it only contains requests addressed to the operating system ("open a file", "draw a window", "send data to the printer"). Without an OS to receive and execute these requests, the application is a dead file on a disk.

This is exactly what happened in Buea: the computer's applications were intact, but with Windows corrupted, the machine could not even start. Similarly, a football game written for Android cannot run on a phone whose Android system is damaged, and a Windows version of a game refuses to install on a machine running only Linux, because it speaks the "language" of the Windows system services.

DID YOU KNOW? **System software in Cameroonian schools and homes**

The most common operating systems in Cameroonian schools and homes are Microsoft Windows (on desktops and laptops) and Android (on smartphones, which are full computers too). Many university laboratories and some cybercafés also use Linux distributions such as Ubuntu, which are free of charge — an advantage where budgets are tight. Antivirus utilities such as Avast or Smadav are popular for scanning flash drives, a major source of viruses in environments where files travel by USB key rather than by Internet.

1.6 Classifying Software: System or Application?

METHOD **Three guiding questions to classify a program**

Faced with any program in an exam or in practice, ask:

1. Whom does it serve? If it mainly serves the computer system itself (managing, maintaining, translating), it is system software. If it serves the end user's task (typing, calculating, playing), it is application software.
2. Can the computer work without it? If removing it stops the machine from functioning properly (OS, driver), it is system software. If the machine still works fine without it (a game, Word), it is application software.
3. Does the user interact with it directly for a personal task? If yes, it is almost always application software. System software mostly works in the background.

WARNING **Common mistake: drivers and utilities are NOT applications**

Many candidates classify device drivers, antivirus programs or disk defragmenters as application software because "the user can open them". This is wrong: their purpose is to maintain and manage the system, not to accomplish a user task like writing a letter. By the three guiding questions above, drivers and utilities are system software. Note also that a language translator (compiler, interpreter) is system software, even though programmers use it directly.

WORKED EXAMPLE 1 **Classify and justify**

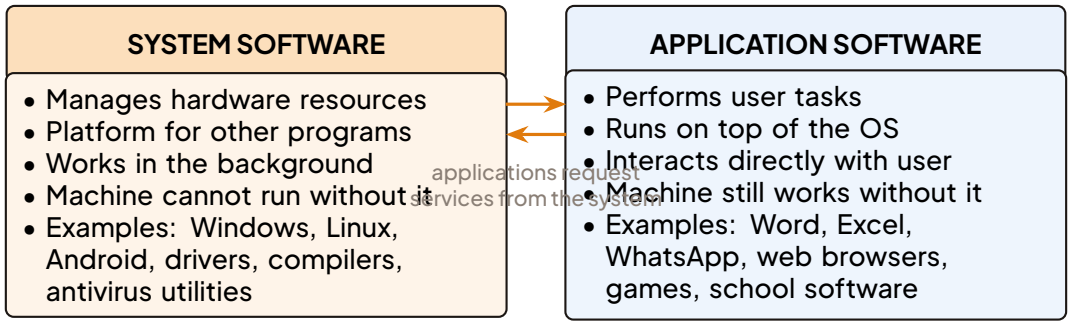
Statement. Classify each of the following as system software or application software, justifying each answer in one line: (a) Ubuntu Linux; (b) Microsoft Excel; (c) a printer driver; (d) a disk cleanup tool; (e) a C compiler; (f) WhatsApp.

SOLUTION

Solution.

- (a) System software: Ubuntu is an operating system; it manages the hardware and provides the platform for other programs.
- (b) Application software: Excel performs a user task (spreadsheets, calculations) and runs on top of an OS.
- (c) System software: a printer driver lets the OS communicate with the printer; it serves the machine, not a user task.
- (d) System software: a disk cleanup tool is a utility; it maintains the storage system.
- (e) System software: a compiler is a language translator; it converts source code into machine code, a service to the system of program production.
- (f) Application software: WhatsApp serves the user directly (messaging) and requires an OS (Android, iOS, Windows) to run.

1.7 System Software vs Application Software: the Exam Comparison



Side-by-side comparison of the two great families of software.

METHOD **Exam tip: "State two differences between system and application software"**

This is one of the most frequent GCE questions on this topic. Prepare a comparison table in advance and always answer with paired differences (one point on each side), for example:

1. System software manages and controls hardware resources, whereas application software performs specific tasks for the user.
2. System software is essential for the computer to function, whereas the computer can

still operate without any particular application.

3. System software generally runs in the background, whereas application software is used interactively by the user.

Give one example of each family to earn full marks.

EXERCISE 1 **Check your understanding**

1. Define system software and give three examples.
2. Explain, using the layered model, why a computer that has "lost its Windows" cannot run Microsoft Word even though Word's files are still on the disk.
3. A friend claims that an antivirus is application software "because you click on it". Using the three guiding questions, explain why this reasoning is wrong.
4. State two differences between system software and application software, giving one example of each.

SOLUTION **Exercise 1**

1. System software is the set of programs that control, coordinate and manage the hardware resources of a computer and provide a platform for application software. Examples: Windows 11, Ubuntu Linux, Android (also accepted: drivers, compilers, antivirus utilities).
2. In the layered model, applications sit above the operating system and never access hardware directly. Word can only run by requesting services (memory, files, screen) from Windows. If Windows is corrupted, those requests have no receiver: Word's files remain on the disk, but the program cannot be loaded or executed.
3. Applying the guiding questions: (1) an antivirus serves the system (detecting and removing threats), not a personal user task; (2) the computer works without it, but its role is maintenance of the system, like other utilities; (3) although the user can open it, its main work is done in the background. Therefore it is utility software, a category of system software. Clicking on a program does not make it an application; its purpose decides.
4. Any two paired differences, e.g.: system software manages hardware resources while application software performs user tasks; system software is indispensable to the machine while applications are optional. Example of system software: Windows; example of application software: Microsoft Word.

KEY POINTS **Summary of the section**

- Software is the intangible part of the computer; it falls into two families: **system software** and **application software**.
- System software manages hardware resources and provides the platform on which applications run; the computer is organised in layers: hardware → OS → utilities/translators → applications → user.
- The main categories of system software are: operating systems, language translators (assemblers, compilers, interpreters), loaders and linkers, device drivers, and utility software.
- The operating system is the resource manager: it schedules the CPU, allocates memory, organises storage and controls input/output devices.
- No application can run without system software: applications only contain requests addressed to the OS.

2 SYSTEM AND UTILITY SOFTWARE

2.1 =

From System to Application: The Translator Family

In the previous section, we discovered that the operating system is only one part of the system software. But how does a program written in, say, C++ become a running application? The answer lies in translators, linkers and deeper system software. We start with those.

2.2 Language Translators

2.3 Assemblers, Compilers and Interpreters

2.4 Linkers and.

.¸Those who write the programs (linkers and loaders). We start with those..

.¸Those who write the programs (linkers and/). Those who write the programs (linkers and loaders). Those who write the programmers (translators). Those whose languages (translators). Those who write the programmers (translators). Those who write the programmers (trans translators). Those. Those whose languages (trans translators). Those who write the programmers (translators). Those. those translators). Those who write the translators. Those who write the translators). Those who translators. Those translators. Those translators. Those translators. Those translators. Those translators. those translators. those translators. Those translators. Those. Those. Those translators. those translators. Those. Those translators. those trans-

lators. Those. those translators. those translators. Those. Those translators. Those. Those. those translators. Those. translators. Those. translators. Those. translators. those. translators. Those. translators. Those. translators. translators. translators. Those. translators. that. Those. translators. translators. Those. translators. translators. translators. those. translators. translators. those. those translators. translators. translators. those. translators. translators. those translators. translators. translators. translators. those. translators. translators. those. translators. translators. translators. translators. translators. translators. translators. translators. those. translators. translators. translators. translators. translators. translators. translators. translators. translators. translators. those. translators. translators. translators. translators. translators. translators. translators. translators. those. translators. translators. translators. translators. translators. translators. translators. translators. translators. translators

one or two sentences.

PROPERTY **The Five Core Functions of an Operating System**

Every operating system performs five core functions:

1. **Process management**: creating, scheduling, suspending and terminating processes (programs in execution), and allocating CPU time among them.
2. **Memory management**: allocating main memory (RAM) to processes, protecting one process's memory from another, and freeing memory when a process ends.
3. **File management**: organising data into files and directories on storage devices, and controlling creation, deletion, reading, writing and access rights.
4. **Device management**: controlling input/output devices (keyboard, printer, disk, network card) through device drivers, and handling interrupts.
5. **Security and user interface**: authenticating users (login/password), enforcing access permissions, and providing the command-line or graphical interface through which the user communicates with the machine.

(The statement of these functions is examinable; no proof is required.)

EXEMPLE **The OS at work when you print a document**

When a student in Buea clicks Print in a word processor: the OS receives the request (user interface), keeps the word processor running while the print job is prepared (process management), reserves memory for the print data (memory management), reads the document file from disk (file management), sends the data to the printer through its driver (device management), and checks that the user is allowed to use that printer (security). One click, five functions.

3.2 Historical Evolution of Operating Systems

Operating systems did not appear overnight. They evolved step by step, each generation solving the main problem of the previous one.

1. The no-OS era and manual operation (1940s–early 1950s). The first computers (vacuum-tube machines) had no operating system. The programmer booked the whole machine, loaded a program manually using switches and punched cards, ran it, and collected the output. The CPU sat idle most of the time while the human prepared the next job.

2. Batch processing (1950s–1960s). To reduce wasted CPU time, operators collected many **jobs** (programs plus their data, on punched cards or magnetic tape) and fed them to the computer in groups (batches), one after another, using a small resident monitor program. There was no interaction between the user and the machine while the job ran: the user submitted the job and came back later for the printed results. The weakness: the CPU still waited idle during slow

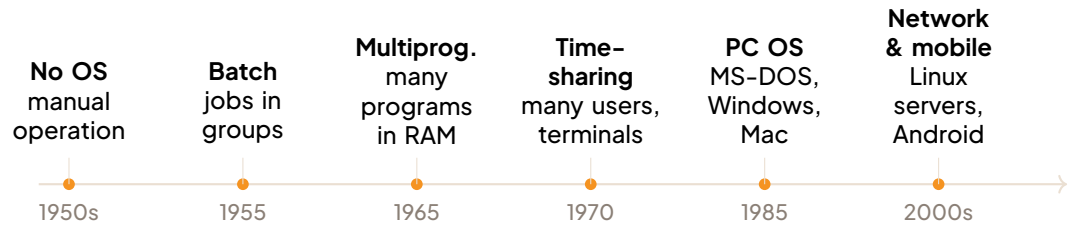
input/output operations.

3. Multiprogramming (1960s). The idea: keep several programs in memory at once. When the running program waits for input/output, the CPU immediately switches to another program. CPU utilisation rises dramatically. This is the birth of true **multitasking** (the ability to interleave the execution of multiple programs on a single processor).

4. Time-sharing (late 1960s–1970s). Multiprogramming was extended so that many users at terminals could use the same computer simultaneously and interactively. The CPU switches so fast between users (each receiving a small **time slice**) that every user has the illusion of having the machine alone. Famous example: UNIX, created at Bell Labs, ancestor of Linux.

5. Personal computer operating systems (1980s–1990s). With cheap microcomputers came single-user operating systems: MS-DOS (command line), then Microsoft Windows and Apple's Mac OS with graphical user interfaces (GUI). One user, one machine, but several programs at once (multitasking).

6. Network, mobile and embedded operating systems (1990s–today). Computers became connected: network operating systems (Windows Server, Linux servers) manage resources shared across machines. Then came the smartphone revolution: Android and iOS. Finally, tiny embedded operating systems run inside ATMs, decoders, car engines and medical equipment.



Timeline of the evolution of operating systems, from manual operation to mobile and network systems.

DID YOU KNOW? **Did you know?**

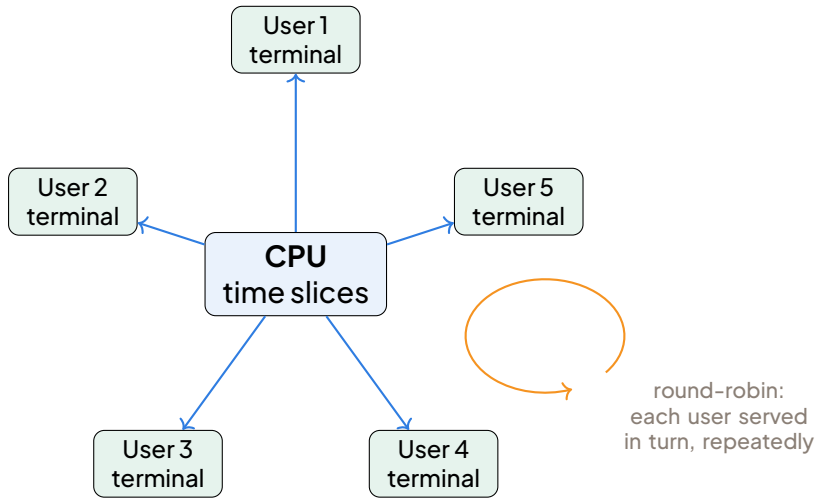
The word batch survives today in computing language: a batch file (`.bat`) in Windows is still a script of commands executed one after another without user interaction — a direct descendant of the batch processing era of the 1960s.

3.3 Main Types of Operating Systems

We now study each major type, with its principle, its strengths and a concrete example.

Batch operating systems. Jobs with similar needs are collected and processed in groups, automatically, one after the other, without any user interaction during execution. The user submits the job and collects results later. Batch processing is still used today for repetitive bulk work: processing the monthly salaries of civil servants, generating end-of-term report cards for a whole school, or nightly bank transaction settlement.

Time-sharing (multi-user) operating systems. Many users connect to one powerful computer through terminals (or over a network). The CPU divides its time into very small slices (a few milliseconds each) and serves the users in turn, in a round-robin fashion. Because the switching is extremely fast, each user feels the machine is dedicated to him or her. Example: a university server on which hundreds of students run programs at the same time; Linux and UNIX servers.



Time-sharing: one CPU serves several user terminals in rapid round-robin time slices.

Real-time operating systems (RTOS). These systems must respond to an event within a strict, guaranteed deadline — a late answer is a wrong answer. They are used where lives or safety depend on timing: air traffic control systems, medical devices (heart monitors, infusion pumps), anti-lock braking systems (ABS) in vehicles, industrial robots. Examples: VxWorks, QNX, real-time variants of Linux.

Network and distributed operating systems. A network OS manages resources shared between connected machines: files, printers, user accounts, security (examples: Windows Server, Linux servers running a school network). A distributed OS goes further: it makes a collection of connected computers appear to the user as one single powerful machine, sharing processing and storage transparently.

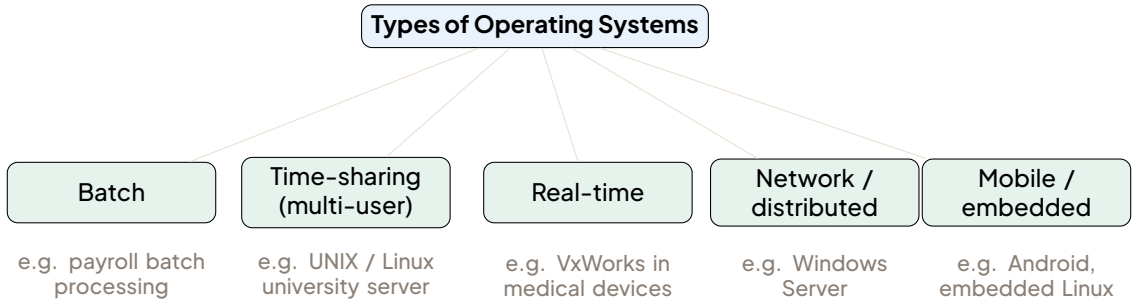
Mobile and embedded operating systems. Mobile OS run on smartphones and tablets: **Android** (open-source, based on the Linux kernel) and **iOS** (Apple, proprietary). Embedded OS are built inside a device to perform a dedicated function: embedded Linux runs in many ATMs, satellite decoders, smart TVs and point-of-sale terminals found in shops across Cameroon.

Popular desktop and server operating systems. For personal computers and servers, the three dominant families are: **Microsoft Windows** (proprietary, the most widespread on desktops), **macOS** (proprietary, Apple computers only), and **Linux** with its many distributions such as Ubuntu, Debian and Fedora (open-source, dominant on servers and free to use and modify).

PROPERTY **Open-source versus proprietary operating systems**

An **open-source** OS (e.g., Ubuntu Linux) is distributed with its source code: anyone may

study, modify and redistribute it, usually free of charge. A **proprietary** OS (e.g., Windows, macOS, iOS) is owned by a company: users buy a licence to use it but may not see or modify the source code.



Classification tree of operating system types, with one example of each.

WARNING **Common mistake: multi-user is not multitasking**

Do not confuse **multi-user** (time-sharing) with **multitasking**. Multitasking means one computer executes several programs at the same time — even a single-user PC with Windows multitasks. Multi-user means several different users use the same computer at the same time, each with his own session and files. A home laptop is multitasking but usually single-user; a university Linux server is both multitasking and multi-user.

METHOD **Choosing an OS type for a given scenario**

To answer a scenario question (a favourite GCE format), proceed in four steps:

1. Identify the number of users: one user (PC/mobile OS) or many simultaneous users (time-sharing/network OS)?
2. Identify the timing constraints: must responses be guaranteed within a deadline? If yes ⇒ real-time OS.
3. Identify the interaction mode: no interaction, bulk repetitive jobs ⇒ batch; interactive use ⇒ time-sharing or PC OS.
4. Identify the device: smartphone/tablet ⇒ mobile OS; dedicated machine (ATM, decoder) ⇒ embedded OS; connected machines sharing resources ⇒ network OS.

Examples: a cybercafé's shared server ⇒ time-sharing/network OS (Linux); a bank's central server ⇒ network OS with strong security; a smartphone ⇒ mobile OS (Android);

hospital monitoring equipment ⇒ real-time OS.

WORKED EXAMPLE 2 **Identifying types of operating systems**

Statement. For each situation below, state the type of operating system best suited and justify your answer: (a) A system controlling the traffic lights at a busy junction in Douala. (b) A server allowing 300 students of a university to run their programming practicals simultaneously. (c) The monthly processing of electricity bills for all customers of a town.

SOLUTION

Solution.

- (a) Real-time OS: the lights must change within strict, guaranteed time limits; a delayed response could cause accidents.
- (b) Time-sharing (multi-user) OS, e.g., a Linux server: many users are served interactively at the same time, each receiving CPU time slices.
- (c) Batch OS: the bills are similar jobs with no need for user interaction; they are collected and processed together as a group, typically overnight.

EXERCISE 2 **Types and evolution of operating systems**

1. Define an operating system and state any four of its core functions.
2. Arrange these stages of OS evolution in chronological order: time-sharing, batch processing, mobile OS, multiprogramming, no-OS era.
3. A GCE candidate writes: "Windows 11 on my home computer is a multi-user operating system because I can open many windows." Explain the error.
4. Give one example each of: an open-source OS, a proprietary OS, and a real-time application.

SOLUTION **Exercise: Types and evolution of operating systems**

1. An OS is the set of system programs that manages the hardware resources of a computer and provides an interface between users/applications and the hardware. Functions (any four): process management, memory management, file management, device management, security and user interface.

2. No-OS era → batch processing → multiprogramming → time-sharing → mobile OS.
3. The candidate confuses multitasking with multi-user. Opening many windows means several programs run at once for one user: that is multitasking. Multi-user means several different users share the same machine simultaneously, as on a time-sharing server.
4. Open-source: Ubuntu Linux. Proprietary: Microsoft Windows (or macOS, iOS). Real-time application: air traffic control, a heart monitor, or an ABS braking system.

KEY POINTS **Key points to remember**

- The OS is the resource manager and intermediary between users and hardware; its five core functions are process, memory, file, device and security/interface management.
- Evolution: no-OS → batch → multiprogramming → time-sharing → PC OS → network/mobile/embedded OS.
- Batch = grouped jobs, no interaction; time-sharing = many users, CPU slices; real-time = guaranteed deadlines; network = shared resources across machines; mobile/embedded = phones and dedicated devices.
- Exam tip: GCE papers frequently ask you to "describe two types of operating systems and give an example of each". Prepare at least three types with one example each, and never confuse multi-user with multitasking.

4 OPERATING SYSTEM INTERFACES

In the previous section, we traced the evolution of operating systems from the early batch-processing machines to modern multi-user, multitasking systems such as Windows, Linux and Android. One striking feature of this evolution is the way human beings talk to the machine. In the 1960s, a computer operator typed cryptic lines of text on a teletype; today, a student in Yaoundé taps icons on a smartphone screen or simply speaks to it. The part of the operating system that makes this communication possible is called the **user interface**. This section defines the concept, studies the two great families of interfaces — the **Command-Line Interface** (CLI) and the **Graphical User Interface** (GUI) — and gives you the practical command skills that the GCE examination expects.

4.1 What is a User Interface?

Intuition first.

Imagine you enter a restaurant in Douala. You do not walk into the kitchen to cook your own meal; you communicate your wishes to the kitchen through a waiter. The waiter is your interface with the kitchen. In the same way, you never manipulate the processor, the memory or the disk directly: you express your wishes ("open this file", "copy this document", "show me my photos") through a layer of the operating system designed for human communication.

DEFINITION **User Interface**

The **user interface** of an operating system is the means by which a user communicates with the operating system: it is the collection of mechanisms (text commands, menus, icons, windows, gestures, voice) through which the user issues instructions to the computer and receives feedback from it. The two main families are:

- the **Command-Line Interface** (CLI), in which the user types textual commands;
- the **Graphical User Interface** (GUI), in which the user manipulates graphical objects (windows, icons, menus) with a pointing device.

WARNING **Interface vs. operating system**

Do not confuse the interface with the operating system itself. The interface is only the visible surface. Windows and Ubuntu Linux both offer a GUI, yet they are different operating systems; conversely, the same Linux system can be used through a GUI or through a pure text terminal.

4.2 The Command-Line Interface (CLI)

Observation.

When a network technician configures a server in a Cameroonian ISP, you will often see a black window full of white text, with no mouse pointer at all. This is a CLI: the user types a command, presses `Enter`, and the system executes it and replies in text. The program that reads and interprets these commands is called the **shell** (examples: `cmd.exe` and PowerShell on Windows; `bash` on Linux).

DEFINITION **Command-Line Interface**

A **Command-Line Interface** is a text-based user interface in which the user types commands, following a strict syntax, at a **prompt**; the operating system executes each command and displays the result as text.

EXEMPLE **A first CLI session (Linux)**

At the prompt `student@pc: $`, the user types:

- `pwd` → the system answers `/home/student` (the current directory);
- `mkdir GCE` → creates a folder named `GCE`;
- `cd GCE` → moves into that folder;
- `ls` → lists the folder's content (empty for now).

Four short lines of text accomplished what would take several mouse clicks in a GUI.

Advantages and disadvantages of the CLI.

PROPERTY **Strengths and weaknesses of the CLI**

- Advantages: (i) speed — an experienced user performs tasks faster by typing than by navigating menus; (ii) low resource consumption — a CLI needs very little memory and processing power, ideal for old machines or headless servers; (iii) scripting — commands can be saved in a file (a **script**) and replayed automatically, which is essential for repetitive tasks; (iv) remote administration — CLI works over low-bandwidth connections (SSH) where a GUI would be unusable.
- Disadvantages: (i) commands and their syntax must be memorised; (ii) a single typing error (a syntax error) makes the command fail; (iii) it is intimidating and unfriendly for beginners; (iv) no visual feedback for complex operations (e.g., drag-and-drop).

4.3 The Graphical User Interface (GUI)

Observation.

Switch on any school computer running Windows: you see a desktop with small pictures (icons), a bar at the bottom, and rectangular areas (windows) that you can move, resize and close by clicking. You do not need to remember any command. This is the GUI, and its four building blocks are summarised by the acronym **WIMP**.

DEFINITION **Graphical User Interface and WIMP**

A **Graphical User Interface** is a user interface in which the user interacts with the operating system through graphical elements manipulated with a pointing device. Its four characteristic components form the acronym **WIMP**:

- W — **Windows**: rectangular areas on the screen, each holding a program or document;

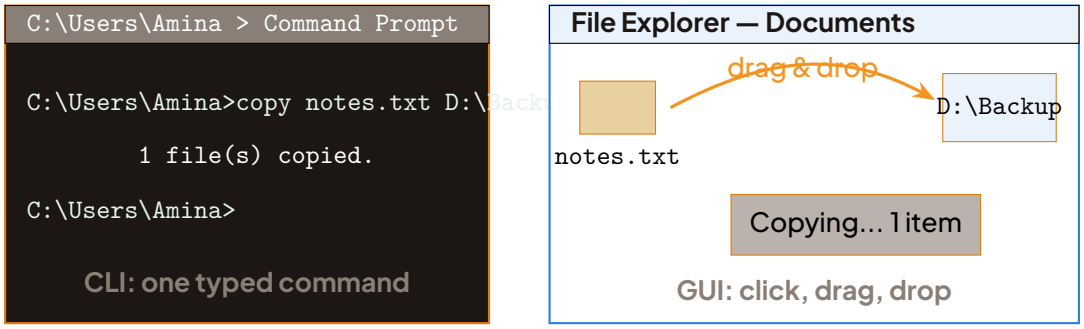
- I — **Icons**: small pictures representing files, folders or programs;
- M — **Menus**: lists of commands that drop down when clicked;
- P — **Pointer**: the on-screen arrow (or finger position) controlled by a mouse, touch-pad or touch screen.

PROPERTY **Strengths and weaknesses of the GUI**

- Advantages: (i) ease of use — intuitive, little or no memorisation required, ideal for beginners; (ii) actions are discoverable through menus and icons; (iii) visual feedback (drag-and-drop, progress bars) reduces errors; (iv) multitasking visibility — multiple windows can be seen and arranged simultaneously.
- Disadvantages: (i) high resource consumption — the graphics layer requires significant memory, processing power and disk space; (ii) repetitive tasks are slow (many clicks); (iii) automation is difficult compared with scripting; (iv) not suitable for headless/remote servers without graphical stack.

DID YOU KNOW? **Where did the GUI come from?**

The GUI with windows, icons and a mouse was pioneered at the Xerox PARC research centre in the 1970s (Xerox Alto), then popularised by the Apple Macintosh (1984) and Microsoft Windows (1985 onward). Today, the same WIMP ideas live on in the touch screens of the Android phones used all over Cameroon: the "pointer" has simply become your finger, and windows become full-screen apps.



The same task — copying `notes.txt` to a backup folder — done in a CLI (left) and in a GUI (right).

4.4 Other Interfaces (Extension)

DID YOU KNOW? **Extension: menu-driven, touch and voice interfaces**

Beyond CLI and GUI, several other interface styles exist. They are not the core of the syllabus but are excellent material for open GCE discussion questions.

- Menu-driven interfaces: the user selects options from numbered lists, e.g. an ATM screen or a mobile-money menu (`*126#` style on MTN/Orange Cameroon). Easy to use, but slow when menus are deep.
- Touch interfaces: a GUI adapted to fingers — tapping, swiping, pinching — as on smartphones and tablets (Android, iOS).
- Voice interfaces: the user speaks commands (e.g. virtual assistants like Google Assistant, Siri). Very natural and accessible (helpful for visually impaired users), but sensitive to noise and accents.

A good exam answer can note that modern systems combine several interfaces: a smartphone offers touch and voice on top of a GUI.

4.5 Essential CLI Commands: Windows and Linux

The GCE expects you to know a small but precise vocabulary of commands in both environments. The following table gives the equivalences, and the method box fixes the five you must master perfectly.

Task	Windows CMD	Linux shell (bash)
List files and folders	`dir`	`ls`
Change directory	`cd Documents`	`cd Documents`
Show current directory	`cd` (alone)	`pwd`
Create a directory	`mkdir GCE`	`mkdir GCE`
Copy a file	`copy a.txt b.txt`	`cp a.txt b.txt`
Delete a file	`del a.txt`	`rm a.txt`
Remove empty directory	`rmdir dir`	`rmdir dir`
Show network configuration	`ipconfig`	`ip addr` (or `ifconfig`)
Clear the screen	`cls`	`clear`

METHOD **Five essential CLI commands every student must master**

Learn the syntax and one example of each; write them from memory in the exam.

1. List the contents of a folder — Windows: `dir`; Linux: `ls`. Example: `ls` displays `notes.txt  GCE  photo.jpg`.

2. Change directory — both systems: `cd <folder>`. Example: `cd GCE` enters the folder `GCE`; `cd ..` goes back up one level.
3. Show where you are — Linux: `pwd` (print working directory). Example: `pwd` answers `/home/student/GCE`. Windows: type `cd` alone.
4. Create a directory — both systems: `mkdir <name>`. Example: `mkdir Revision` creates the folder `Revision`.
5. Copy and delete files — Windows: `copy source destination` and `del file`; Linux: `cp source destination` and `rm file`. Example: `cp notes.txt Backup/` copies `notes.txt` into the folder `Backup`.

WARNING **Danger: `del` and `rm` delete permanently**

Files deleted with `del` (Windows CMD) or `rm` (Linux) do not go to the Recycle Bin: they are removed immediately and are very difficult to recover. Always check the file name twice before pressing `Enter`. A classic disaster is `rm` typed in the wrong directory — run `pwd` or `ls` first to confirm where you are. On Linux, `rm -rf` is especially dangerous (recursive force delete).

4.6 Hands-On Practice: Navigating in CMD and in a Linux Terminal

EXPERIMENT **Guided practice: building a revision folder**

Perform this sequence twice: once in Windows CMD, once in a Linux terminal (or an emulator such as Termux on a phone).

1. Open the terminal and display the current directory (`cd` alone on Windows; `pwd` on Linux). Note the answer.
2. List the files present (`dir` / `ls`).
3. Create a folder: `mkdir GCE_CS`.
4. Enter it: `cd GCE_CS`, then create two sub-folders: `mkdir Notes` and `mkdir PastPapers`.
5. Verify with `dir` / `ls` that both folders exist.
6. Go back up: `cd ..`, then delete the folder `PastPapers` (Windows: `rmdir PastPapers`; Linux: `rmdir PastPapers`) and confirm its disappearance with a final listing.

You have just performed, in text, exactly what a file explorer does with the mouse.

4.7 CLI or GUI? Choosing the Appropriate Interface

The two interfaces are not enemies; each is appropriate in a given context. Copying one file is fastest in a GUI (drag and drop); copying five hundred files every night is fastest as a CLI script that runs automatically.

WORKED EXAMPLE 3 **Comparing CLI and GUI for the same task**

Statement. A secretary must (a) copy one letter file to a USB key, and (b) copy every evening 300 report files from a folder `Reports` to a backup disk. For each task, state which interface (CLI or GUI) is more appropriate and justify.

SOLUTION

Solution.

- (a) One file, once: the GUI is more appropriate. The task is occasional and simple; drag-and-drop requires no memorised syntax and gives visual confirmation. Opening a terminal would be slower here.
- (b) 300 files, every evening: the CLI is more appropriate. A single command such as `cp Reports/* /media/Backup/` (Linux) or `copy Reports\*.* D:\Backup\` (Windows) copies all files at once, and it can be saved in a script and scheduled to run automatically every evening. Doing 300 drag-and-drop operations daily in a GUI would be slow and error-prone.

Conclusion: GUI for occasional, simple, visual tasks; CLI for repetitive, large-scale or automated tasks.

KEY POINTS **Exam tip: CLI vs. GUI**

Be ready to state, without hesitation, two advantages and two disadvantages of the CLI compared with the GUI. Safe answers:

- CLI advantages: fast for experienced users; uses few resources; allows scripting/automation; works over low-bandwidth remote connections.
- CLI disadvantages: commands must be memorised; syntax errors are easy for beginners; no visual feedback for complex operations.
- GUI advantages: easy and intuitive; little memorisation; visual feedback reduces errors; good for multitasking visibility.
- GUI disadvantages: consumes more memory and processing power; slow for repetitive tasks; difficult to automate; not suitable for headless servers.

Structure your answer as full sentences, e.g. “Unlike a GUI, a CLI consumes very little memory because it displays only text.”

EXERCISE 3 **Checking your mastery**

1. Define the term user interface and name its two main families.
2. Expand the acronym WIMP and explain each component in one sentence.
3. Give the Windows CMD command and the Linux command to: (a) list files; (b) create a folder called `Exam`; (c) delete a file `old.txt`; (d) show current directory.
4. State two advantages and two disadvantages of a CLI compared with a GUI.
5. A school administrator must rename and reorganise 2 000 student files every term. Which interface do you recommend, and why?
6. (Challenge) Explain why a system administrator managing a remote web server in a data centre would prefer a CLI over a GUI.

SOLUTION **Exercise: Checking your mastery**

1. The user interface is the means by which the user communicates with the operating system, issuing instructions and receiving feedback. The two main families are the Command-Line Interface (CLI) and the Graphical User Interface (GUI).
2. Windows (rectangular screen areas holding programs/documents), Icons (small pictures representing files or programs), Menus (clickable lists of commands), Pointer (on-screen indicator moved by the mouse or finger).
3. (a) Windows: `dir`; Linux: `ls`. (b) Both: `mkdir Exam`. (c) Windows: `del old.txt`; Linux: `rm old.txt`. (d) Windows: `cd` (alone); Linux: `pwd`.
4. Advantages of CLI: fast for experienced users; low memory/processor consumption; commands can be scripted and automated; works over SSH with low bandwidth. Disadvantages: commands and syntax must be memorised; a typing error causes failure, so it is unfriendly to beginners; no visual feedback for complex operations.
5. A CLI, because the task is repetitive and large-scale: the commands can be written once in a script and replayed each term, saving time and avoiding the errors of thousands of manual GUI operations.
6. A remote web server typically runs headless (no monitor/keyboard). CLI via SSH uses minimal bandwidth and resources, allows scripting of routine maintenance, and does not require a graphical stack to be installed and running on the server.

Having mastered how we communicate with an operating system, the next step is to learn how an operating system is installed on a machine and how it organises files on storage devices — the subject of the following section.

5 OS INSTALLATION AND FILE STORAGE

Having explored how we interact with the operating system through various interfaces in the previous section, we now turn to the practical steps of getting that operating system onto the machine in the first place, and how it organises the data we entrust to it. Installing an OS is a rite of passage for every computer scientist; understanding file storage is the daily bread of system administration. In this section, we will walk through the complete installation workflow, demystify disk partitioning and file systems, and master the hierarchy of files and directories.

5.1 Preparing for Installation

Before inserting any installation media, a prudent technician performs a series of checks. Neglecting these steps is the most common cause of failed installations or, worse, permanent data loss.

DEFINITION **Hardware Requirements**

Every operating system publishes **minimum** and **recommended** hardware specifications. Minimum requirements guarantee the OS will boot; recommended requirements guarantee a usable, responsive experience. Key parameters include:

- CPU architecture: 64-bit (x86_64 / ARM64) is standard for modern OSes (Windows 10/11, Ubuntu 20.04+). 32-bit support is largely deprecated.
- RAM: 4 GB minimum for light use; 8 GB+ recommended for multitasking.
- Storage: 20–64 GB free space for the OS partition (system files, page file, updates). SSD strongly preferred over HDD for performance.
- Firmware: UEFI (Unified Extensible Firmware Interface) is the modern standard replacing legacy BIOS. Secure Boot capability is often required for Windows 11.
- Windows 11 Specifics: TPM 2.0 (Trusted Platform Module) and a DirectX 12 compatible GPU / WDDM 2.x driver are mandatory.

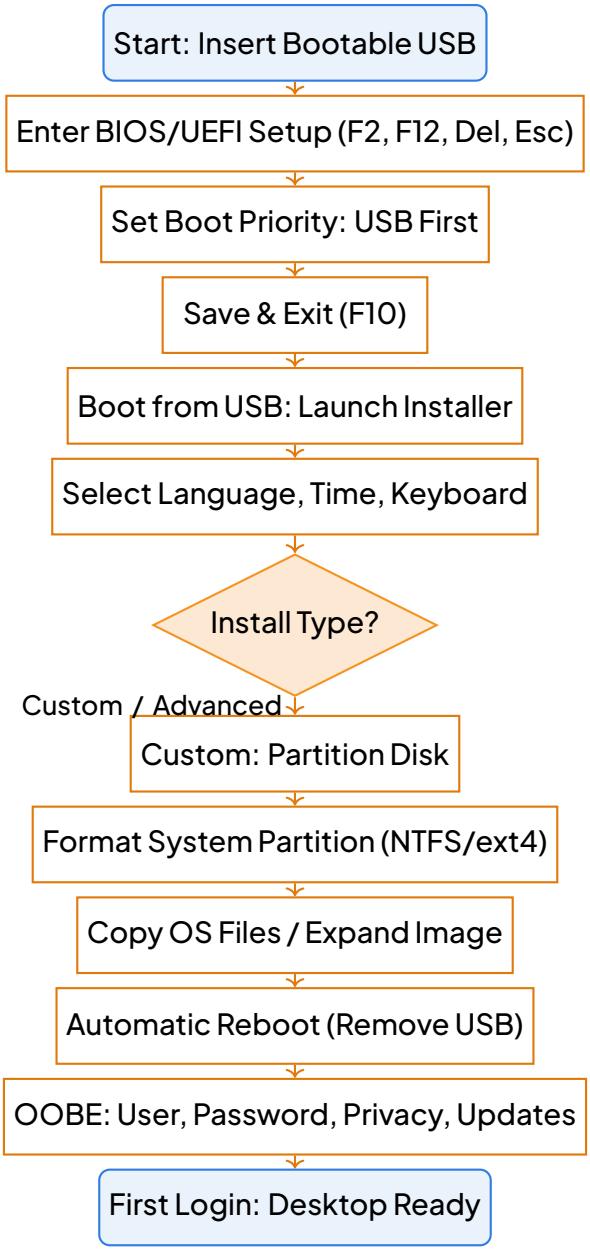
WARNING **Back Up Before You Begin**

Installation involves writing to the disk partition table and formatting partitions. This destroys all existing data on the target partitions. Always back up important documents, photos, and configuration files to an external drive or cloud storage before starting. A Cameroonian proverb says: “He who does not know where he comes from will not know where he is going.” In IT: “He who does not back up will eventually lose data.”

METHOD **Creating a Bootable USB Key (e.g., with Rufus)**

Most modern installations use a USB flash drive (minimum 8 GB). The ISO image of the OS must be written to the USB in a way that makes it bootable.

1. Download the official ISO file (e.g., Windows 11 from Microsoft, Ubuntu LTS from Canonical).
2. Insert a USB flash drive (warning: all data on it will be erased).
3. Launch `Rufus` (Windows) or `BalenaEtcher` / `dd` command (Linux/macOS).
4. Select the USB device and the ISO file.
5. Partition scheme: Choose `GPT` for UEFI systems (modern PCs); `MBR` for legacy BIOS.
6. Target system: `UEFI (non CSM)` for GPT; `BIOS (or UEFI-CSM)` for MBR.
7. File system: `FAT32` (required for UEFI boot) or `NTFS` (Windows-only installs on legacy BIOS).
8. Click `START` and wait for completion.



Flowchart: Typical OS Installation Process from Bootable USB to First Login

5.2 BIOS/UEFI and Boot Order

The firmware (BIOS or UEFI) is the first code that runs when the computer powers on. It performs the **POST** (Power-On Self-Test) and then looks for a bootable device according to the **boot order** (boot priority list).

PROPERTY **UEFI vs Legacy BIOS Boot**

- UEFI + GPT: Modern standard. Supports disks > 2 TB, faster boot, Secure Boot (malware protection), graphical setup menus. Requires partition table type `GPT` and an `EFI System Partition (ESP)` formatted FAT32 (typically 100–500 MB).
- Legacy BIOS + MBR: Older standard. Limited to 4 primary partitions (or 3 primary + 1 extended), max disk size 2 TB. Boots from the Master Boot Record (first 512 bytes of disk).
- CSM (Compatibility Support Module): A UEFI feature that emulates Legacy BIOS for older OSes or bootloaders. Disabling CSM forces pure UEFI boot.

Exam Tip: GCE questions often ask to identify the firmware mode from a screenshot (graphical mouse-driven menu = UEFI; text-only blue/grey menu = Legacy BIOS) or to match partition scheme to firmware (GPT ↔ UEFI, MBR ↔ BIOS).

To boot from the USB, you must enter the firmware setup (common keys: `F2`, `F10`, `F12`, `Del`, `Esc`) and move "USB Storage Device" or the specific USB model name to the top of the Boot Priority list. On UEFI systems with Secure Boot enabled, you may need to disable Secure Boot temporarily for some Linux distributions, though Ubuntu and Fedora now support Secure Boot out of the box. For Windows 11, Secure Boot and TPM 2.0 must be enabled.

5.3 Disk Partitioning: Organising the Physical Disk

A physical disk (HDD or SSD) is just a large array of addressable blocks. **Partitioning** divides this space into logical sections, each appearing to the OS as a separate "disk" with its own drive letter (Windows) or mount point (Linux).

DEFINITION **Partition Types**

- Primary Partition: A partition described directly in the partition table (max 4 on MBR; unlimited on GPT). Can be marked `Active` (bootable) on MBR.
- Extended Partition: A special primary partition (MBR only) that acts as a container for Logical Partitions. Allows more than 4 partitions on MBR disks. Only one extended partition per disk.
- Logical Partition: A partition inside an extended partition. Cannot be bootable (Active) on MBR.
- EFI System Partition (ESP): Contains the UEFI bootloaders. On UEFI/GPT: a small (100–500 MB) FAT32 partition flagged `esp` and `boot`. Mandatory for UEFI boot.
- Microsoft Reserved Partition (MSR): A small (16 MB) partition reserved by Windows on GPT disks for internal use (e.g., dynamic disk conversion).
- System Partition (Windows term): The partition containing the bootloader (`bootmgr`, `BCD`). On UEFI this is the ESP; on Legacy BIOS it is the "System Reserved" partition

(NTFS, 500 MB) marked Active.

- Boot Partition (Windows term): The partition containing the OS files (`C:\Windows`). Often the same as the System Partition on simple setups.
- OS Partition (C: / root /): Holds the operating system files, programs, and page/swap file.
- Data Partition (D: / /home): Holds user files. Separating data from OS simplifies backups and OS reinstallation without touching user data.
- Recovery Partition: Hidden partition containing a factory restore image or Windows RE (Recovery Environment).
- Swap Partition (Linux): Dedicated partition for virtual memory (swap space), formatted as `swap`.

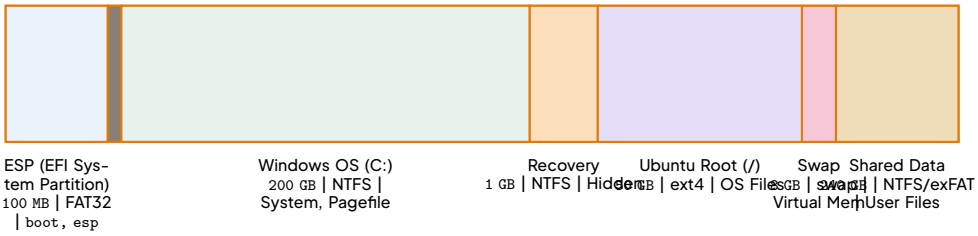


Diagram: Partitioned Hard Disk showing System, Swap, Recovery and Data Partitions (Dual-boot Windows/Ubuntu example)

5.4 Formatting and File Systems

Creating a partition only defines its boundaries. **Formatting** (making a file system) writes the metadata structures (tables, journals, bitmaps) that allow the OS to store and retrieve files.

PROPERTY **Comparison of Common File Systems**

Feature	FAT32	NTFS	exFAT	ext4
Max File Size	4 GB	16 TB (practically)	16 EB	16 TB – 1 EB
Max Volume Size	2 TB (32 GB format limit in Windows)	256 TB	128 PB	1 EB
Permissions / ACLs	No	Yes (Advanced)	No	Yes (POSIX + ACLs)
Journaling	No	Yes	No	Yes
Native OS Support	All (Win, Mac, Linux, Consoles)	Windows (Read-only Mac/Linux default)	Win, Mac, Linux (modern kernels)	Linux (Windows via WSL/Ext2Fsd)
Typical Use Case	USB sticks ≤ 32 GB, UEFI ESP	Windows System Drive (C:), Internal HDDs	Large USB drives (>32 GB), SDXC cards, Cross-platform external drives	Linux System Drive (/), /home, Servers

KEY POINTS **Choosing the Right File System**

- Windows System Drive: `NTFS` (mandatory for security, compression, encryption, large files).
- Linux System Drive: `ext4` (standard, journaling, performance).
- UEFI Boot Partition (ESP): `FAT32` (firmware requirement).
- External USB/HDD for Windows + Mac + Linux: `exFAT` (no 4 GB file limit, wide support).
- Legacy USB ≤ 32 GB for old devices (TV, car stereo): `FAT32`.

5.5 Installation Steps Overview: Windows and Ubuntu

While the graphical wizards differ, the logical sequence is identical.

EXPERIMENT **Windows 11 Installation Walkthrough (Key Decision Points)**

1. Boot from USB → Language/Keyboard → `Install Now`.
2. Product Key: Enter or `I don't have a product key` (activate later).
3. Edition: Select `Windows 11 Pro` (or Home/Education).
4. License Terms: Accept.
5. Installation Type: `Custom: Install Windows only (advanced)`. Never choose "Upgrade" on a clean disk.
6. Where to install: Critical Step. You see a list of partitions.
   - If disk is new/unallocated: Select `Unallocated Space` → `New` → `Apply`. Windows creates: `System (ESP)`, `MSR`, `Primary (C:)`, `Recovery`. Select the large `Primary` partition → `Next`.
   - If dual-booting: Do not delete Linux partitions (ext4/swap). Select the existing NTFS partition for Windows (or create new in free space).
7. Copying files / Features / Updates: Automatic (10–30 min). Machine reboots multiple times. Remove USB after first reboot to avoid re-entering installer.
8. OOBE (Out-of-Box Experience): Region, Keyboard, Network (connect for updates/account), Account (Microsoft account or Offline "Domain join instead" → Local account), Privacy settings.
9. Desktop: Check `Settings → Windows Update → Check for updates` → Install drivers via `Device Manager` or manufacturer website.

EXPERIMENT **Ubuntu 22.04/24.04 LTS Installation Walkthrough**

1. Boot from USB → GRUB menu → `Try or Install Ubuntu`.
2. Language / Keyboard / Wireless: Connect to Wi-Fi for updates/drivers during install.
3. Installation Type: `Normal installation` (recommended) + `Download updates` + `Install third-party drivers` (Wi-Fi, GPU).
4. Installation Type (Disk):
   - `Erase disk and install Ubuntu` → Automatic partitioning (LVM optional).
   - `Something else` (Manual) → Required for Dual Boot.
     - Select free space → + → Size (e.g., 30–50 GB) → Type: Primary → Location: Beginning → Use as: `ext4` → Mount point: `/` → OK.
     - Select free space → + → Size (RAM size or 4–8 GB) → Use as: `swap area` → OK.

- Bootloader device: Select the physical disk (e.g., `/dev/sda` or `/dev/nvme0n1`), NOT a partition number.

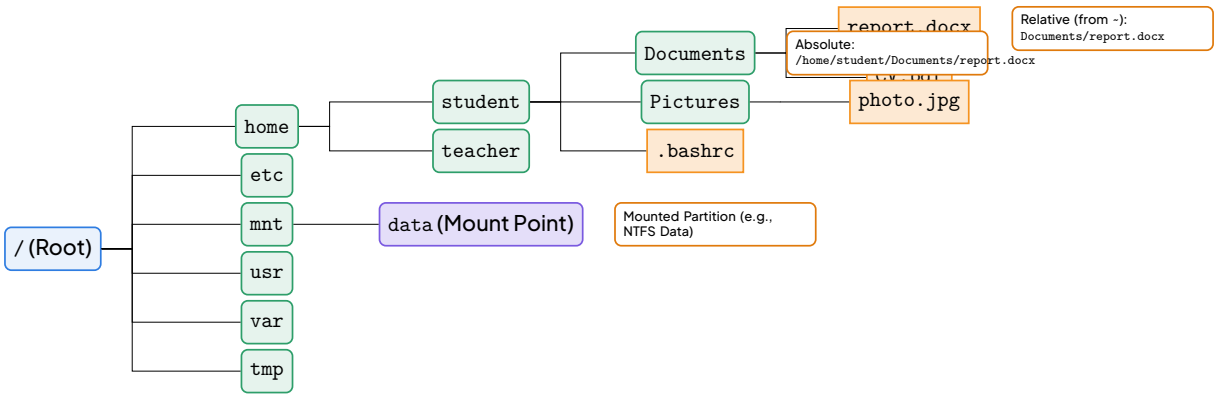
5. Where are you? Timezone (e.g., Africa/Douala).
6. Who are you? Name, Computer name, Username, Password, `Require my password to login` (recommended), `Encrypt my home folder` (optional, adds complexity).
7. Installation proceeds → `Restart Now` → Remove USB → Login.

5.6 File Storage Concepts: The Logical View

Once the OS is installed, it presents a logical namespace over the physical partitions. Understanding this namespace is fundamental to scripting, programming, and system administration.

DEFINITION **File System Hierarchy**

- File: A named collection of related data stored on secondary storage. Identified by a **filename** and often a **file extension** (e.g., `report.docx`, `script.py`, `image.png`). The extension hints at the file type/content but does not enforce it on Linux (magic numbers do); on Windows it determines default association.
- Folder / Directory: A special file containing a table mapping names to inodes (Linux) or MFT entries (Windows). Allows hierarchical organisation.
- Root Directory: The top of the hierarchy.
  - Windows: Each partition has its own root, denoted by a drive letter + backslash: `C:\`, `D:\`.
  - Linux: Single unified tree. Root is `/`. Other partitions are **mounted** onto directories (mount points) under root (e.g., `/home`, `/mnt/data`).
- Absolute Path: Full path from the root to the file. Unique.
  - Windows: `C:\Users\Student\Documents\report.docx`
  - Linux: `/home/student/Documents/report.docx`
- Relative Path: Path from the **current working directory (CWD)**. Uses `.` (current dir) and `..` (parent dir).
  - If CWD is `/home/student`: `Documents/report.docx` or `./Documents/report.docx`.
  - To go up: `../Pictures/photo.jpg`.
- Home Directory: User's personal folder. Windows: `C:\Users\<username>`. Linux: `/home/<username>` (often abbreviated as `~`).



File System Hierarchy Tree showing Absolute and Relative Paths (Linux style)

WARNING **Path vs Extension Confusion**

- Path: Locates the file in the hierarchy (`C:\Users\A\file.txt`).
- Extension: The suffix after the last dot in the filename only (`.txt`).
- Common Mistake: Writing "The extension is `C:\Users\A\file.txt`".
- Common Mistake: Thinking `.bashrc` has extension `bashrc`. It is a hidden file (starts with dot) with no extension.
- Windows Default: Hides extensions for known file types. Always enable "File name extensions" in File Explorer View tab to avoid executing `malware.pdf.exe` thinking it is a PDF.

5.7 File Management Operations

These are the bread-and-butter operations. Mastery of both GUI and CLI (Command Line Interface) is expected at GCE Advanced Level.

KEY POINTS **Core File Operations**

Operation	Windows (GUI / CLI)	Linux (CLI)
Create File	Right-click → New / `type nul > file.txt`	`touch file.txt` / `cat > file.txt`
Create Folder	Right-click → New Folder / `mkdir dir`	`mkdir dir` / `mkdir -p a/b/c`
Copy	Ctrl+C / Ctrl+V / `copy src dst` / `xcopy /E`	`cp src dst` / `cp -r src dst`
Move / Rename	Drag-drop / F2 / `move src dst` / `ren old new`	`mv src dst` (handles both)
Delete	Del (Recycle Bin) / Shift+Del (Perm) / `del` / `rmdir /s`	`rm file` / `rm -r dir` / `rm -rf` (DANGEROUS)
Search	Explorer Search Box / `dir /s /b *pattern*`	`find /path -name "pattern"` / `locate pattern`
Attributes	Right-click → Properties → Hidden, Read-only / `attrib +h +r file`	`chmod 400 file` (read-only) / `ls -la` (dot files hidden)

WORKED EXAMPLE 4 **Interpreting a Directory Listing**

Statement: A student runs `ls -la` in their home directory on Ubuntu and sees:

- `total 48`
- `drwxr-xr-x 5 student student 4096 Oct 10 10:00 .`
- `drwxr-xr-x 3 root root 4096 Oct 9 12:00 ..`
- `-rw-r--r-- 1 student student 220 Oct 9 12:00 .bash_logout`
- `-rw-r--r-- 1 student student 3771 Oct 9 12:00 .bashrc`
- `drwxr-xr-x 2 student student 4096 Oct 10 09:30 Documents`
- `drwxr-xr-x 2 student student 4096 Oct 10 09:30 Downloads`
- `-rw-r--r-- 1 student student 0 Oct 10 10:00 notes.txt`
- `drwxr-xr-x 2 student student 4096 Oct 10 09:50 Projects`

Answer the following:

1. What do the entries `.` and `..` represent?
2. Identify all hidden files/directories and explain why they are hidden.
3. What are the permissions (in symbolic and octal) for the `Documents` directory? Can

`student` create a file inside? Can `teacher` (member of `student` group? No, other) read a file inside?

4. Write the absolute path of `notes.txt`.
5. Write the relative path from `Projects` to `notes.txt`.

SOLUTION

Detailed Solution:

1. `.` represents the current directory (`/home/student`). `..` represents the parent directory (`/home`).
2. Hidden entries start with a dot (`.`). Here: `.bash_logout`, `.bashrc`, and the directory entries `.` and `..` themselves. They are hidden because their names begin with `.`; `ls` without `-a` ignores them. They typically store user configuration.
3. `Documents` line: `drwxr-xr-x`.
   - `d` = directory.
   - `rwx` (owner `student`): read, write, execute (enter directory). Octal `7`.
   - `r-x` (group `student`): read, execute. Octal `5`.
   - `r-x` (others): read, execute. Octal `5`.
   - Octal: 755.

   Can student create file? Yes, owner has write (`w`) permission on the directory. Can teacher (other) read a file inside? `teacher` falls under "others". Others have `r-x` on the directory. `x` allows `cd` into directory and accessing inodes of known filenames. `r` allows listing names (`ls`). So `teacher` can `ls Documents` and `cat Documents/known_file.txt` if the file itself grants read permission to others.
4. Absolute path: `/home/student/notes.txt`
5. Relative path from `Projects` (sibling of `notes.txt`): `../notes.txt`

5.8 Post-Installation Good Practices

The installation is not finished at the desktop. A professional ensures the system is secure, updated, and organised.

METHOD **Post-Installation Checklist**

1. Run Updates Immediately:

- Windows: `Settings → Windows Update → Check for updates`. Repeat until "You're up to date". Install optional driver updates.
- Ubuntu: `sudo apt update && sudo apt full-upgrade -y`. Reboot if kernel updated.

2. Install/Verify Drivers:
   - Windows: `Device Manager` (Win+X → M). Look for yellow triangles. Download GPU (NVIDIA/AMD/Intel), Chipset, Audio, LAN/Wi-Fi drivers from laptop manufacturer website (not generic chip vendor) for best power management.
   - Ubuntu: `Software & Updates → Additional Drivers`. Select proprietary NVIDIA driver if gaming/ML. `sudo ubuntu-drivers autoinstall`.
3. Security Baseline:
   - Windows: Windows Defender is excellent; ensure `Virus & threat protection` is On. Enable `Ransomware protection → Controlled folder access`.
   - Ubuntu: `sudo ufw enable` (firewall). Install `clamav` for scanning shared drives. Keep `unattended-upgrades` enabled (default).
4. Organise User Folders:
   - Move `Documents`, `Downloads`, `Pictures` to a separate Data partition (D: or `/mnt/data`) to survive OS reinstall. Windows: Right-click folder → Properties → Location → Move. Linux: Bind mounts in `/etc/fstab` or symlinks.
5. Create a System Image / Backup:
   - Windows: `Control Panel → Backup and Restore (Windows 7) → Create a system image` (to external drive).
   - Linux: `Timeshift` (snapshots) or `Clonezilla` (full disk image).
6. Install Essential Software: Browser (Firefox/Chrome), Archive tool (7-Zip / PeaZip), Editor (VS Code / Notepad++), Terminal (Windows Terminal / Terminator), PDF Reader.

DID YOU KNOW? **Cameroon Context: The "Cyber" Reality**

In many Cameroonian cybercafés and school labs, you will encounter machines running outdated Windows 7 or 10 versions, often without updates due to expensive or metered internet (MTN/Orange data bundles). As a future technician, you must know how to:

- Create a local WSUS Offline Update medium to patch machines without internet.
- Use Clonezilla or `dd` to deploy a master image to 20 identical lab PCs in 20 minutes.
- Configure `/etc/apt/sources.list` to use a local mirror (e.g., `mirror.univ-yaounde1.cm` if

available) to save bandwidth.

- Explain to a client why their 128 GB USB stick formatted as FAT32 cannot hold their 5 GB video file (File system limit!), and reformat it to exFAT.

5.9 Consolidation Exercises

EXERCISE 4 **GCE Style: Ordering Installation Steps**

Arrange the following steps in the correct logical order for a clean Windows 11 installation on a new NVMe SSD (UEFI/GPT):

1. Create user account and configure privacy settings.
2. Boot from USB and select "Custom: Install Windows only".
3. Enter BIOS/UEFI and set USB as first boot device.
4. Windows creates partitions (System, MSR, Primary, Recovery) automatically.
5. Select "Unallocated Space" and click "New", then "Apply".
6. Remove USB after first automatic reboot.
7. Copy Windows files, install features, install updates (multiple reboots).
8. Select Language, Time, Keyboard.

SOLUTION **Exercise 1**

Correct order: 3 → 8 → 2 → 5 → 4 → 7 → 6 → 1. Key reasoning: BIOS config (3) precedes boot. Language (8) is first installer screen. Partitioning (5,4) happens before file copy (7). USB removal (6) happens after first reboot triggered by file copy, before OOBE (1).

EXERCISE 5 **File System Choice Scenario**

A student has a 64 GB USB flash drive. They want to:

1. Store a 10 GB virtual machine disk image (`.vdi`).
2. Use the drive on Windows 11 lab PCs, a MacBook Pro (macOS), and a Raspberry Pi 4 (Ubuntu Server).
3. Be able to set read-only attribute on sensitive folders to prevent accidental deletion.

Which file system should they choose? Justify your answer by comparing FAT32, NTFS, and exFAT against the three requirements.

SOLUTION **Exercise 2**

Choice: exFAT. Justification:

- Req 1 (10 GB file): FAT32 fails (max 4 GB). NTFS and exFAT support >10 GB.
- Req 2 (Cross-platform): NTFS is read-only on macOS by default (requires third-party driver like Paragon/NTFS-3G for write). exFAT has native read/write support on Windows, macOS, and Linux (kernel 5.4+). FAT32 works everywhere but fails Req 1.
- Req 3 (Attributes/Permissions): Critical Trap. exFAT does not support POSIX permissions or Windows ACLs. It only supports the basic "Read-only" flag (like FAT32). NTFS supports full ACLs. However, since Req 1 and 2 eliminate FAT32 and NTFS respectively, exFAT is the only viable compromise. The student must rely on OS-level write-protection (e.g., physical write-protect switch on SD card adapter, or making files read-only in each OS individually) rather than persistent filesystem metadata.

EXERCISE 6 **Path Resolution**

Given the Linux directory tree from the figure earlier (`/home/student/Documents/report.docx`, `/home/student/Pictures/photo.jpg`, `/mnt/data/Shared`). Assume the Current Working Directory (CWD) is `/home/student/Projects`. Write the relative path to reach:

1. `report.docx`
2. `photo.jpg`
3. `Shared` (mounted at `/mnt/data/Shared`)
4. The root directory `/`

SOLUTION **Exercise 3**

1. `../Documents/report.docx` (Up to student, down to Documents)
2. `../Pictures/photo.jpg` (Up to student, down to Pictures)
3. `../../mnt/data/Shared` (Up to student, up to home, up to root, down to mnt, data, Shared)

4. `../../..` (Up three levels: Projects → student → home → /)

KEY POINTS **Key Takeaways for GCE Revision**

- Installation Pre-reqs: Check specs (TPM 2.0, Secure Boot for Win11), BACKUP, Create Bootable USB (Rufus: GPT/UEFI vs MBR/BIOS).
- Boot Process: POST → Firmware (BIOS/UEFI) → Boot Order → Bootloader (GRUB / Windows Boot Manager) → Kernel → Init/Userland.
- Partitioning: GPT (UEFI) allows many partitions + ESP (FAT32). MBR (Legacy) max 4 primary (use Extended/Logical for more). Separate OS and Data partitions.
- File Systems: NTFS (Win System), ext4 (Linux System), FAT32 (ESP, tiny USB), exFAT (Large Cross-platform USB). Know the 4 GB FAT32 limit!
- Paths: Absolute (from Root) vs Relative (from CWD). `.` = here, `..` = parent. Windows uses \, Linux uses /.
- Post-Install: Updates → Drivers → Security (Firewall/AV) → Backup Strategy → Organise Folders.

6 INTEGRATION ACTIVITY

6.1 Aims of the activity

This integration activity closes the chapter by asking you to use – not merely recall – everything you have studied about system software. By the end of the activity, you should be able to:

- distinguish **system software** problems from hardware or application software problems when diagnosing a faulty machine;
- select the appropriate **utility software** (antivirus, disk defragmenter, disk cleanup, backup tool) or system-level action (driver installation, OS reinstallation) for a given fault;
- justify the choice of an **operating system** type for a school context, taking into account cost, licensing and ease of maintenance;
- compare **CLI** and **GUI** interfaces and use basic command-line instructions for file management;
- plan a complete **OS installation**: BIOS/UEFI boot order, **partitioning** scheme, **file system** choice and formatting;
- design a logical **file storage** organisation using folders and absolute paths.

KEY POINTS **Skills mobilised**

Diagnosis → decision → planning → implementation → justification. At the GCE Advanced Level, examiners reward a candidate who not only gives a correct technical answer but justifies it with the vocabulary of the chapter: utility, driver, partition, file system, boot sequence, path.

6.2 Situation

EXPERIMENT **Reviving the computer room of GBHS – a system software mission**

Government Bilingual High School (GBHS) Nkol-Eton has a computer room containing 10 desktop machines bought eight years ago. Each machine has a 500 GB hard disk, 4 GB of RAM and a dual-core processor. The room is used by Upper Sixth candidates for their practical work and by the school secretary to keep digital copies of examination files. The computer science teacher, Mr. Tabi, has been transferred, and for one full year nobody maintained the room. The new principal asks your class to prepare the room before the GCE practical session. Your team inspects the machines and records the following inventory:

Machines	Observed symptoms	Quantity
Group A	Machine does not start: after power-on, the screen displays `No bootable device found`. One machine shows the error only since a student changed settings in the BIOS setup.	3
Group B	Machine boots into Windows but takes more than 10 minutes; pop-up windows appear, the homepage of the browser has changed, and the antivirus icon is missing. Students confirm they use personal flash drives daily.	4
Group C	Machine works correctly, but all files (exam papers, student projects, photos) are dumped on the Desktop and in a single folder called `STUFF`. Nothing can be found quickly.	3

Additional data collected by your team:

- The school owns 10 genuine Windows licences and has also received a free Ubuntu Linux installation DVD from a partner NGO.
- One Group A machine shows the `No bootable device` message even after the BIOS boot order is corrected; its hard disk is detected by the BIOS but contains no valid operating system.
- The school secretary needs a clear storage space where each class's examination files can be saved and retrieved by absolute path.
- The school has a limited Internet connection, so heavy downloads must be avoided.

Your mission, working in teams of four, is to diagnose the ten machines, restore them to working order, and present a complete, justified maintenance plan to the principal.

6.3 Tasks

Task 1 – Diagnosis (mobilises Lessons 38 and 39).

For each group (A, B, C), answer the following:

1. State whether the problem is related to system software, application software or hardware, and justify your answer.
2. Identify the precise system software component involved (operating system, driver, utility, BIOS settings).
3. Explain why the Group B symptoms strongly suggest a malware infection, and name the category of utility software required.

Task 2 – Choice of corrective actions (mobilises Lessons 39 and 40).

1. For the Group A machine whose BIOS boot order was changed, propose the simplest corrective action and explain why no reinstallation is needed.

2. For the Group A machine with no valid OS on the disk, justify a full OS reinstallation and choose between Windows and Ubuntu for the school, giving two arguments (licence cost, user familiarity, maintenance).
3. For Group B, write the ordered sequence of utility-based actions to clean the machines without reinstalling the OS, and explain why reinstalling immediately would be a waste of time.
4. For Group C, explain why the problem is not a software fault but an organisational one, and name the OS tool used to solve it (file manager / CLI).

Task 3 – Installation checklist (mobilises Lesson 42).

Choose one Group A machine and write the complete ordered installation checklist, including:

1. the BIOS/UEFI step: how to enter the setup and how to set the boot order so that the machine starts from the installation medium (DVD or bootable flash drive);
2. a partitioning plan for the 500 GB disk with at least two partitions (system and data), giving the size of each partition and a justification;
3. the file system chosen for each partition (e.g. NTFS) with one justification;
4. the post-installation steps: driver installation, antivirus installation, creation of user accounts, and Windows/Linux updates.

Task 4 – File storage design (mobilises Lesson 42).

1. Design a folder structure for the secretary's machine so that examination files are organised by year, then by class, then by subject.
2. Write the absolute path of the file `paper1.docx` belonging to Upper Sixth, Computer Science, year 2025, assuming the data partition is `D:` (Windows) or `/data` (Linux).
3. Give two rules of good file management that the school should display in the computer room.

Task 5 – Command-line demonstration (mobilises Lesson 41).

1. On a real machine or a simulator, demonstrate three CLI commands for file management: creating a directory, moving/renaming a file, and listing the contents of a directory. Give the exact syntax in Windows (`cmd`) or Linux (bash).
2. State one advantage of the CLI over the GUI for a technician managing many machines, and one advantage of the GUI for an ordinary student.

Task 6 – Presentation.

Each team presents its maintenance plan in 10 minutes. Every technical choice (OS, partition sizes, file system, utility) must be justified orally. The class votes for the most rigorous plan.

6.4 Model answer

WORKED EXAMPLE 5 **Task 1 – Diagnosis**

State the nature of each group's problem and the component involved.

SOLUTION

Group A (no boot): system software problem. The message `No bootable device found` means the BIOS/UEFI cannot find a disk containing a bootable operating system. For the machine whose BIOS settings were changed, the component involved is the BIOS boot order (firmware configuration). For the other machine, the disk has no valid OS installed: the missing component is the operating system itself.
Group B (slow, pop-ups, changed homepage): system software environment corrupted by malware introduced through students' flash drives. The missing/misused component is the antivirus utility; the slowness also calls for disk cleanup and defragmentation utilities. It is not a hardware fault: the machines boot and run.
Group C (disorganised files): neither hardware nor software fault – the OS works. It is a file management (organisation) problem, solved with the file manager (GUI) or CLI commands.

WORKED EXAMPLE 6 **Task 2 – Corrective actions**

Propose and justify the action for each group.

SOLUTION

Group A, BIOS-changed machine: enter the BIOS setup (press `Del` or `F2` at power-on), restore the hard disk as first boot device, save and exit. No reinstallation is needed because the OS on the disk is intact; only the boot sequence was wrong.
Group A, empty-disk machine: full OS reinstallation. Choice: Windows, because the school already owns 10 genuine licences (no extra cost) and students and the secretary are familiar with its GUI, reducing training time. (Ubuntu would be acceptable: it is free and virus-resistant, but would require retraining users.)
Group B: ordered cleanup sequence:

1. disconnect the machine from any network and scan each flash drive;
2. install (or reinstall) an antivirus from a clean medium and update its database;
3. run a full system scan and quarantine/delete the malware;
4. run Disk Cleanup to remove temporary files, then Defragment the hard disk;
5. uninstall unknown programs and reset the browser;
6. create a policy: all flash drives must be scanned before use.

Reinstalling immediately would waste time and licences because the OS itself is func-

tional; utilities can restore it.
Group C: reorganise files using File Explorer or CLI commands (`mkdir`, `move`); no software repair is required.

WORKED EXAMPLE 7 **Task 3 – Installation checklist for one Group A machine**

Write the ordered checklist with partitioning plan.

SOLUTION

1. Back up any recoverable data from the disk (if any).
2. Power on and press `Del`/`F2` to enter the BIOS/UEFI setup.
3. Set the boot order: (1) DVD-ROM / USB flash drive, (2) hard disk. Save and exit (`F10`).
4. Insert the Windows installation medium; the machine boots from it.
5. Choose language, then Custom installation.
6. Partition the 500 GB disk:
   - Partition C: 150 GB, formatted NTFS – for the OS and programs. Justification: 150 GB is ample for Windows plus utilities, and separating the system from data means a future reinstallation of C: will not destroy students' files.
   - Partition D: 350 GB, formatted NTFS – for data. Justification: NTFS supports large files, file permissions (security between users) and is the native Windows file system.
7. Install Windows on C: and wait for completion; remove the medium when prompted.
8. Restore the BIOS boot order so that the hard disk is first (faster, safer boot).
9. Install the drivers (chipset, graphics, network, sound) from the manufacturer's media – without drivers the hardware cannot be fully exploited by the OS.
10. Install the antivirus and update it.
11. Create user accounts: one administrator (teacher) and standard accounts (students, secretary).
12. Run system updates, then test: reboot, open programs, check both partitions.

WORKED EXAMPLE 8 **Task 4 – File storage design**

Design the folder structure and give absolute paths.

SOLUTION

Folder structure on the data partition:

```
D: \EXAMS
    • \2024   \2025
        • \LOWER-SIXTH   \UPPER-SIXTH
            • \CSC   \MATH   \PHY

                    D:\EXAMS\2025\UPPER-SIXTH\CSC\paper1.docx
```

Linux equivalent: `/data/EXAMS/2025/UPPER-SIXTH/CSC/paper1.docx`.
Two rules to display in the room: (1) every file must be saved in the folder of its year, class and subject – never on the Desktop; (2) file names must be meaningful (e.g. `CSC_seq3_2025.docx`), without spaces or vague names like `doc1`.

WORKED EXAMPLE 9 **Task 5 – CLI demonstration**

Give three file-management commands and compare CLI and GUI.

SOLUTION

Windows (`cmd`) version:

- `mkdir D:\EXAMS\2025` – creates the directory `2025` inside `EXAMS`.
- `move D:\STUFF\paper1.docx D:\EXAMS\2025\UPPER-SIXTH\CSC\` – moves the file to its correct folder.
- `dir D:\EXAMS\2025` – lists the contents of the directory.

Linux (bash) equivalents: `mkdir -p /data/EXAMS/2025`, `mv paper1.docx /data/EXAMS/2025/UPPER-SIXTH/CSC/`, `ls /data/EXAMS/2025`.
Advantage of the CLI: commands can be written once into a script and executed on many machines automatically – ideal for a technician maintaining ten computers. Advantage of the GUI: it is intuitive (icons, drag-and-drop) and requires no memorised syntax – ideal for an ordinary student.

WARNING **Common mistakes to avoid in your presentation**

- Reinstalling the OS as the answer to every fault: a wrong BIOS boot order or a virus infection is fixed faster with the correct setting or utility.
- Forgetting to restore the hard disk as first boot device after installation.
- Putting the OS and the data on the same partition without justification.
- Writing relative paths when an absolute path (starting from the drive letter or root) is requested.
- Confusing a driver (lets the OS control hardware) with a utility (maintains the system).

EXERCISE 7 **Extension – securing the room for the future**

The principal wants to prevent the Group B disaster from happening again. Propose a written computer room policy of five rules combining technical measures (antivirus updates, restricted user accounts, flash-drive scanning) and behavioural rules for students. Justify each rule with one concept from this chapter.

SOLUTION **Extension exercise**

Expected elements (any five, justified):

1. Standard (non-administrator) accounts for students – prevents unauthorised software installation; exploits the OS's user-management feature.
2. Antivirus installed, updated and scheduled to scan weekly – a utility can only detect malware it knows; updates keep the database current.
3. Mandatory scanning of every flash drive before use – flash drives were the infection vector; the antivirus utility blocks threats at entry.
4. No personal software or games on school machines – reduces malware sources and disk clutter.
5. Monthly preventive maintenance (disk cleanup, defragmentation, updates) by a trained student team supervised by the teacher – utilities keep the system efficient over time.
6. Regular backup of the `D:\EXAMS` folder to an external disk – the backup utility protects examination files against disk failure.

Marking guide: 1 mark per rule, 1 mark per correct justification using chapter vocabulary (utility, account, backup, update, scan).

DID YOU KNOW?

Why this matters in Cameroon

Many Cameroonian schools, like GBHS Nkol-Eton in this activity, run computer rooms with ageing machines and no full-time technician. GCE practical examinations require every candidate to find a working, well-organised machine. The skills you have just mobilised – diagnosis, safe cleanup, clean installation, partitioning and disciplined file storage – are exactly those of a school-level IT technician, a real employment opportunity for a science graduate in Yaoundé, Douala, Bamenda or Buea.

7 METHODS AND WORKED EXAMPLES

7.1 Methods and Worked Examples

METHOD **Classifying Software into System, Application and Utility Categories**

1. Identify the purpose: Does the software manage hardware resources and provide a platform for other software? → **System Software** (OS, drivers, firmware, language translators).
2. Does it help the user perform a specific non-computing task? → **Application Software** (Word processor, spreadsheet, browser, payroll, media player).
3. Does it perform maintenance, optimisation or security tasks on the system itself? → **Utility Software** (Antivirus, disk defragmenter, backup tool, file compressor, disk cleaner).
4. Check for overlap: Some suites bundle utilities (e.g. Windows Defender is a utility inside the OS). Classify the core function of the named program.
5. Write the justification: Always quote the defining characteristic (resource management vs user task vs system maintenance).

WORKED EXAMPLE 10 **Software Classification in a School Computer Lab**

Statement: The ICT department of Government Bilingual High School Mbankomo has just received new computers. The technician installs the following programs:

1. Windows 11 Pro
2. Microsoft Office 2021 (Word, Excel, PowerPoint)
3. VLC Media Player
4. EaseUS Todo Backup (for system image backup)
5. Device driver for the HP LaserJet Pro MFP M428 printer
6. 7-Zip (file archiver)
7. Python 3.11 Interpreter

Classify each item as System Software, Application Software or Utility Software. Justify each classification.

SOLUTION

Detailed Solution:
We apply the method: identify the primary purpose of each program.

Software	Classification	Justification
Windows 11 Pro	System Software (Operating System)	Manages hardware resources (CPU, RAM, disks), provides UI, loads applications, handles file system. It is the platform.
Microsoft Office 2021	Application Software	Allows users to create documents, spreadsheets, presentations — specific user tasks unrelated to running the computer itself.
VLC Media Player	Application Software	Plays audio/video files for the user's entertainment/education. It is a user-facing task.
EaseUS Todo Backup	Utility Software	Performs a maintenance task: creating/restoring system images to protect data integrity. It acts on the system structure.
HP Printer Driver	System Software (Device Driver)	Translates OS print commands into hardware-specific signals. It extends the OS kernel to manage a specific peripheral.
7-Zip	Utility Software	Compresses/decompresses archives to save disk space and organise files — a system optimisation/maintenance task.
Python 3.11 Interpreter	System Software (Language Translator / Runtime)	Translates high-level code into machine code at runtime. It is a system tool essential for developing/running other software, not a user task per se.

Examiner's Tip: If asked "Is an antivirus system or utility software?", answer: Utility Software. It performs a specific maintenance task (scanning/removing malware) rather than continuously managing resources like the OS kernel. However, modern OSes include antivirus utilities (Windows Defender) as built-in components.

METHOD **Matching Operating System Functions to Management Roles**

1. List the four core management roles: **Process Management**, **Memory Management**, **File Management**, **Device Management** (I/O).
2. Analyse the scenario/action: Identify the resource being allocated, protected, scheduled or organised.
3. Match keywords:
   - Scheduling, PCB, context switch, deadlock, multiprocessing, time-slice, quantum → Process Management.
   - Allocation, paging, segmentation, virtual memory, protection, fragmentation, swap, page fault → Memory Management.
   - Directory, hierarchy, permissions, naming, backup, FAT/NTFS/inode, ACL → File Management.
   - Driver, interrupt handler, buffering, spooling, plug-and-play, I/O request → Device Management.
4. State the specific OS module: e.g. "The Process Scheduler (part of Process Management) decides which process gets the CPU."

WORKED EXAMPLE 11 **Identifying OS Management Roles in Action**

Statement: Describe the specific Operating System management role involved in each of the following situations:

1. The OS moves the data of a minimised program from RAM to the hard disk (swap file) to free space for the active program.
2. The OS prevents a user from deleting a file in the `C:\Windows\System32` folder because they lack "Write" permission.
3. The print spooler holds five print jobs in a queue on the hard disk and sends them to the printer one by one.
4. The OS switches the CPU from running the web browser to running the antivirus scan

after 20 ms.

5. A user plugs in a USB flash drive; the OS detects it, loads the appropriate driver, and assigns it the drive letter `F:`.

SOLUTION

Detailed Solution:
We use the keyword matching method.

1. Memory Management (specifically Virtual Memory / Paging). Reasoning: Moving inactive pages from physical RAM to secondary storage (swap file/page file) to free frames for active processes is the core of virtual memory management. Keywords: "RAM", "hard disk", "free space".

2. File Management (specifically Access Control / Permissions). Reasoning: Enforcing read/write/execute permissions based on user identity (ACLs) to protect system integrity is a file system security function. Keywords: "permission", "delete", "folder".

3. Device Management (specifically Spooling / I/O Buffering). Reasoning: Managing the speed mismatch between CPU and printer by queuing jobs on disk (spool) is a classic I/O management technique. Keywords: "print spooler", "queue", "printer".

4. Process Management (specifically Process Scheduling / Context Switching). Reasoning: The time-slice (quantum) expiry and saving/restoring Process Control Blocks (PCBs) to switch the CPU between ready processes is the scheduler's job. Keywords: "switches CPU", "20 ms", "running".

5. Device Management (specifically Plug-and-Play / Driver Management). Reasoning: Automatic hardware detection, driver loading, and logical device naming (drive letter) are I/O subsystem responsibilities. Keywords: "plugs in", "detects", "loads driver", "drive letter".

Key Distinction: Spooling (Simultaneous Peripheral Operations On-Line) is always Device Management, even though it uses disk space (File Management) and memory (Memory Management). The primary role is managing the peripheral device.

METHOD **Analysing the Evolution of Operating Systems**

1. Identify the Generation/Era: Batch (1950s) → Multiprogrammed Batch (1960s) → Time-Sharing / Multi-user (1970s) → Personal Computer / Single-user (1980s) → Network / Distributed (1990s) → Mobile / Embedded / Real-time (2000s+) → Cloud / Virtualised (Present).

2. Match the **Key Limitation** of the previous era to the **Innovation** of the next:

- Batch: No interaction, CPU idle during I/O → Multiprogramming: Keep multiple jobs in RAM, switch on I/O.
- Multiprogramming: No user interaction → Time-sharing: Rapid context switch (quantum) gives illusion of simultaneous use.
- Mainframe/Time-sharing: Expensive, centralised → PC OS: Cheap, single-user, GUI (Mac OS, Windows).
- Standalone PCs: No sharing → Network OS (Novell, Windows NT): File/print sharing, domain security.
- Desktop OS: Not real-time, heavy → Mobile/RTOS: Preemptive priority scheduling, low latency, power management.

3. For GCE questions: Define the type (Batch, Real-Time, Distributed, etc.), give one advantage and one disadvantage, and cite a typical application.

WORKED EXAMPLE 12 **Evolution: From Batch to Real-Time Systems**

Statement:

1. Define Real-Time Operating System (RTOS). Distinguish clearly between Hard Real-Time and Soft Real-Time systems, giving one practical example of each in a Cameroonian context.
2. Explain why a standard General-Purpose OS (like Windows 11 or standard Linux) is not suitable for controlling the automatic braking system (ABS) of a car.
3. A factory in Douala uses a Batch Processing system for monthly payroll. Explain two reasons why this is still appropriate today, despite the availability of interactive systems.

SOLUTION

Detailed Solution:
(a) Definition and Distinction A **Real-Time Operating System (RTOS)** is an OS designed to process data and respond to events within a strictly defined, guaranteed time constraint (deadline). Correctness depends not only on the logical result but also on the time at which the result is produced.

Characteristic	Hard Real-Time	Soft Real-Time
Deadline Miss	Catastrophic failure (system crash, physical damage, loss of life).	Degraded quality (usable but poor experience), recoverable.
Scheduling	Strict priority-based preemptive; deterministic latency.	Priority-based but may allow some jitter; best effort.
Cameroon Example	ABS Braking System / Airbag Deployment in vehicles on the Douala-Yaoundé highway. Missing the 10 ms deadline causes accident.	Live Video Streaming (Canal 2 / CRTV Web TV) or Mobile Money USSD (*126#). A 2-second delay is annoying but not fatal.

(b) Why GPOS fails for ABS

1. Non-deterministic Scheduling: Windows/Linux use fair-share or Completely Fair Scheduler (CFS) algorithms optimised for throughput and responsiveness, not worst-case latency. A garbage collection pause or background update could delay the brake signal unpredictably.
2. No Priority Inversion Protection: Standard OSes may suffer unbounded priority inversion (low-priority task holds lock needed by high-priority ISR). RTOS uses Priority Inheritance Protocol or Immediate Ceiling Protocol.
3. Memory Management: Virtual memory paging (page faults) introduces unbounded delays. RTOS typically uses flat memory or locked pages (no swapping).
4. Kernel Preemptibility: GPOS kernels are often not fully preemptible (critical sections disable interrupts). RTOS kernels are fully preemptible.

(c) Why Batch Processing remains for Payroll

1. High Volume, Uniform Processing: Payroll involves thousands of identical records (calculate tax, CNPS contributions, net pay) with no user interaction required during the run. Batch systems maximise throughput by eliminating context-switch overhead for interactive handling.
2. Resource Efficiency / Cost: The job can run overnight (off-peak hours) on cheaper hardware or cloud spot instances, utilising 100% CPU without affecting daytime interactive services (HR portal, email). It simplifies auditing: one log file for the entire run.

Exam Tip: "Batch processing is obsolete" is False. It is optimal for repetitive, high-volume, non-interactive tasks (billing, payroll, scientific simulation, end-of-day bank reconciliation).

METHOD **Comparing Operating System Interfaces (CLI vs GUI vs NUI)**

1. Identify the interaction paradigm:
   - **CLI (Command Line Interface)**: Text commands, keyboard-only, scripting/automation, low resource usage, steep learning curve.
   - **GUI (Graphical User Interface)**: WIMP (Windows, Icons, Menus, Pointer), mouse/touch, discoverable, higher resource usage (RAM/GPU).
   - **NUI (Natural User Interface)**: Touch, gesture, voice, gaze, pen; invisible interface, context-aware.
2. Match the task to the best interface:
   - Repetitive admin on 500 servers → CLI (scripts, SSH, PowerShell, Bash).
   - Novice user editing photos → GUI (visual feedback, drag-drop).
   - Surgeon viewing X-rays in sterile room → NUI (gesture/voice, no touch).
   - Visually impaired user → NUI (screen reader/voice) or CLI (keyboard only).
3. Key CLI commands to know (Windows/Linux): `dir/ls`, `cd`, `copy/cp`, `del/rm`, `mkdir`, `ipconfig/ifconfig (ip a)`, `ping`, `tasklist/ps`, `shutdown`.

WORKED EXAMPLE 13 **Choosing and Using OS Interfaces**

Statement: A system administrator at the University of Buea manages a farm of 20 Linux servers (headless, no monitors) and 50 Windows 11 workstations in the student lab.

1. Recommend the primary interface for managing the Linux servers. Justify with two technical reasons.
2. The administrator needs to create a script that runs every night at 2:00 AM to compress the `/var/log` directory into a dated archive `/backup/logs_YYYYMMDD.tar.gz` on the Linux servers. Write the Bash command line (single line) to achieve this compression.
3. A student in the lab wants to find the IP address of their Windows workstation. Give the exact CLI command (Command Prompt or PowerShell) and the GUI path (Settings app navigation) to find it.
4. Explain why CLI is preferred over GUI for the server automation task in (b), referencing resource consumption and reproducibility.

SOLUTION

Detailed Solution:

(a) Primary Interface for Linux Servers: **CLI (Command Line Interface)** via SSH.

1. Resource Efficiency: Servers are "headless" (no GPU/monitor). A GUI (X11/Wayland + Desktop Environment like GNOME/KDE) consumes 500 MB – 2 GB RAM and CPU cycles just to render windows. CLI (Bash over SSH) uses less than 10 MB RAM, leaving maximum resources for actual services (Apache, MySQL, Docker).

2. Automation & Scripting: CLI commands are text streams. They can be chained (pipes), looped, conditioned, and saved as scripts (`.sh`) for unattended execution (cron jobs). GUI actions require manual clicking or fragile GUI-automation tools (Sikuli, AutoIt).

(b) Bash Command for Nightly Backup:

```
tar -czf "/backup/logs_$(date +%Y%m%d).tar.gz" /var/log
```

Explanation: `tar` (tape archive), `-c` create, `-z` gzip compress, `-f` file name. `$(date +%Y%m%d)` executes date command inline to generate YYYYMMDD. Quotes handle spaces in path (good practice).

(c) Finding IP Address on Windows 11:

- CLI: Open Command Prompt (`cmd`) or PowerShell → type `ipconfig` (or `Get-NetIPConfiguration` in PS). Look for `IPv4 Address` under the active adapter (e.g. Ethernet or Wi-Fi).
- GUI Path: `Start > Settings (Win+I) > Network & Internet > [Ethernet / Wi-Fi] > Hardware properties` → scroll to `IPv4 address`.

(d) CLI vs GUI for Automation:

1. Resource Consumption: Running a GUI session (even minimal) on 20 servers 24/7 wastes ≈ 10–20 GB total RAM and significant CPU/GPU. CLI cron job runs for seconds, uses KB of RAM, zero GPU.

2. Reproducibility / Idempotency: A Bash script is a text file (source code). It can be version-controlled (Git), code-reviewed, tested on a staging server, and deployed identically to all 20 servers via Ansible/SSH. A GUI procedure ("Click here, drag there") cannot be version-controlled, differs slightly per screen resolution/theme, and is human-error prone.

Note: GCE exams often ask for specific commands: `ls -l` (long list), `chmod 755` (permissions), `ps aux | grep apache` (find process), `sudo apt update` (package mgmt). Memorise the core 15–20.

METHOD **Steps for Operating System Installation and Disk Preparation**

1. Pre-installation Checks: Hardware compatibility (CPU arch x64/ARM, RAM min, Disk space, Secure Boot/TPM 2.0 for Win 11), Backup data, Obtain ISO + License key.

2. Create Bootable Media: Use `Rufus` (Windows) or `dd` (Linux) to write ISO to USB (FAT32

for UEFI). Verify SHA256 checksum.

3. Configure Firmware (BIOS/UEFI): Boot from USB. Set **UEFI Mode** (not Legacy/CSM) for modern OS (Win 10/11, Ubuntu 20.04+). Disable Secure Boot only if Linux distro requires it (usually keep ON for Windows).
4. Partitioning Scheme (Critical GCE Topic):
   - MBR (Master Boot Record): Legacy, max 4 primary partitions (or 3 primary + 1 extended with logical drives), max disk size 2 TB. Used with BIOS/Legacy boot.
   - GPT (GUID Partition Table): Modern, 128+ partitions, disk size up to 9.4 ZB. Required for UEFI boot. Contains **EFI System Partition (ESP)** (FAT32, 100–500 MB, bootloaders).
5. File System Formatting:
   - Windows: **NTFS** (system), **FAT32/exFAT** (USB/ESP).
   - Linux: **ext4** (system), **swap** (partition/file), **FAT32** (ESP).
   - macOS: **APFS**.
6. Post-Install: Drivers (Chipset, GPU, Network), Updates, Antivirus, User accounts, Backup image (Macrium Reflect, Timeshift).

WORKED EXAMPLE 14 **OS Installation: Partitioning, UEFI, and File Systems**

Statement: A student wants to install Windows 11 on a new 1 TB NVMe SSD in a laptop with UEFI firmware. The student boots from a USB stick created with Rufus.

1. The Rufus "Partition scheme" dropdown offers MBR and GPT. Which must be selected for Windows 11 on UEFI hardware? Explain why the other option would fail.
2. During Windows Setup, the student sees "Drive 0 Unallocated Space". They click "New". Windows creates four partitions. List the name, typical size, file system, and function of each partition.
3. The student wants to dual-boot Ubuntu 22.04 later. They shrink the Windows C: partition by 200 GB in Windows Disk Management, leaving "Unallocated". During Ubuntu installation, they choose "Something else". Describe how to configure the two essential Linux partitions in that 200 GB space (Mount point, File System, Size, Type).
4. Explain the role of the **EFI System Partition (ESP)** in the dual-boot boot process. Where are the `.efi` bootloader files stored for Windows and Ubuntu respectively?

SOLUTION

Detailed Solution:

(a) Partition Scheme: **GPT**. Reason: Windows 11 requires UEFI firmware (mandatory TPM 2.0, Secure Boot, VBS). UEFI only boots from GPT disks (specifically from the ESP on a GPT disk). If MBR is selected, Rufus creates a BIOS/Legacy bootable USB. The UEFI firmware (in UEFI mode) will not detect the USB as a boot option, or if forced to Legacy/CSM mode, Windows 11 Setup will refuse to install ("This PC can't run Windows 11" / "The selected disk has an MBR partition table. On EFI systems, Windows can only be installed to GPT disks").
(b) Four Partitions Created by Windows 11 Setup (GPT/UEFI):

Order	Name (Label)	Size	File System / Function
1	Recovery	500 MB – 1 GB	NTFS. Windows Recovery Environment (WinRE) for "Reset this PC", Startup Repair.
2	EFI System Partition (ESP)	100 MB – 500 MB	FAT32. Stores bootloaders (`/EFI/Microsoft/Boot/bootmgfw.efi`), BITLOCKER keys. No drive letter.
3	MSR (Microsoft Reserved)	16 MB	No FS (Raw). Placeholder for dynamic disk conversion (LDM), GPT sync.
4	Primary (C:)	Remainder (998 GB)	NTFS. OS, Programs, User Data. Boot Partition.

(c) Ubuntu Partitions in 200 GB Unallocated Space:

1. Root Partition (/):
   - Mount point: `/`
   - File System: **ext4** (journaling, standard for Linux root).
   - Size: 180 GB (or all remaining minus swap).
   - Type: Primary (GPT allows many primaries; usually Primary on GPT).
2. Swap Partition:
   - Mount point: `swap area` (no mount point).
   - File System: **swap area**.
   - Size: 4 GB – 20 GB (Rule of thumb: = RAM size if RAM < 16 GB; 4–8 GB if RAM >= 16 GB for hibernation support).
   - Type: Primary.

3. Note on /home: Optional separate partition. If not separate, it resides inside /.
4. ESP: Do NOT create a new ESP. Select the existing Windows ESP (Partition 2) as "EFI System Partition" / Mount point `/boot/efi` without formatting. Ubuntu's `grub-install` will add `/EFI/ubuntu/grubx64.efi` there.

(d) Role of ESP in Dual Boot: The **ESP (EFI System Partition)** is the shared firmware-level boot menu repository.

- Windows Bootloader: `/EFI/Microsoft/Boot/bootmgfw.efi`
- Ubuntu Bootloader (GRUB): `/EFI/ubuntu/grubx64.efi` (and `shimx64.efi` for Secure Boot).

The UEFI firmware NVRAM holds Boot Entries (Boot0000, Boot0001...) pointing to these `.efi` files. GRUB (usually set as default Boot0000) presents a menu: "Ubuntu", "Advanced options", "Windows Boot Manager (on /dev/nvme0n1p2)". Selecting Windows chainloads `bootmgfw.efi`.
Common Mistake: Formatting the ESP during Linux install destroys Windows bootloader. Always select existing ESP, mount at `/boot/efi`, do not format.

METHOD **Analysing File Systems: FAT32, exFAT, NTFS, ext4**

1. Identify the constraints: Max file size, Max volume size, OS compatibility (Read/Write), Features (Journaling, Permissions, Encryption, Compression, Snapshots).
2. Comparison Matrix (Memorise key limits):

Feature	FAT32	exFAT	
Max File Size	4 GB - 1 B	16 EB	
Max Volume Size	2 TB (32 GB format limit in Win)	128 PB	
Journaling	No	No	
Permissions/ACLs	No	No	
Encryption	No	No	
Native OS Support	All (Win, Mac, Linux, Consoles)	All (Modern)	Win (R/W),
Typical Use	USB ≤ 32 GB, Boot (ESP)	USB > 32 GB, SDXC, Cross-OS	Windo

3. Cluster/Block Size: Smaller cluster = less **slack space** (internal fragmentation) for many small files, but larger FAT/MFT. Default: 4 KB (NTFS, ext4), 32 KB+ (FAT32 large vol), 128 KB+ (exFAT large vol).
4. Fragmentation: FAT32/NTFS suffer external fragmentation (need defrag). ext4 uses

delayed allocation and **extents** to minimise it; `e4defrag` exists but rarely needed. SSDs: Trim/Discard replaces defrag.

WORKED EXAMPLE 15 **File System Selection and Limits**

Statement: A media company in Yaoundé handles 4K video files (typical size 15–50 GB per file). They use external drives to transfer footage between Windows 11 editing workstations, a macOS ProRes grading station, and a Linux render farm.

1. Why is FAT32 unsuitable for these drives? Give the specific technical limit.
2. Compare exFAT and NTFS for this scenario. Which is best? Justify with three reasons covering compatibility, file size, and metadata needs.
3. The Linux render farm stores final outputs on a local 20 TB RAID array formatted ext4. Explain two advantages of ext4 over NTFS for this Linux-only high-performance storage.
4. A junior technician formats a 64 GB SD card as NTFS for a Canon EOS R5 camera. The camera reports "Card Error". Explain why, referencing the SD Card Association standard and file system overhead.

SOLUTION

Detailed Solution:
(1) FAT32 Limit: Maximum File Size = 4 GB (precisely 4, 294, 967, 295 bytes = $2^{32} - 1$). Since 4K video files are 15–50 GB, they cannot be stored on FAT32. The copy operation will fail with "File too large for destination file system".
(2) exFAT vs NTFS – Best Choice: **exFAT**.

Criterion	NTFS	exFAT
File Size Support	Unlimited (16 EB) – OK	Unlimited (16 EB) – OK
Cross-OS Compatibility	Windows: Full R/W. macOS: Read-Only (native). Requires paid Paragon/NTFS-3G for write. Linux: R/W via NTFS-3G (FUSE, slower, userspace).	Windows: Full R/W. macOS: Full R/W (native since 10.6.5). Linux: Full R/W (native kernel module since 5.4/5.7).
Metadata / Permissions	Complex ACLs, Ownership, EFS. Causes "Access Denied" errors when moving between OSes (different SIDs/UIDs).	No POSIX permissions/ACLs. Simple "everyone can read/write". Ideal for media shuttle drives where permissions are a hindrance.
Journaling / Reliability	Full Journaling (Metadata). Safer for system drives.	No Journaling. Higher risk of corruption on unsafe eject. Mitigation: Always "Safely Remove".
Overhead	High (MFT, $Bitmap, $LogFile). Wasteful on small flash drives.	Very Low. Designed for flash memory.

Verdict: exFAT is best. Native R/W on all three OSes without drivers, supports huge files, low overhead on flash media. NTFS permissions cause cross-OS headaches; macOS write support is not native.

(3) ext4 Advantages on Linux-only RAID:

1. Native Kernel Implementation / Performance: ext4 is in the Linux kernel (VFS layer). No FUSE userspace overhead (context switches, copy-in/copy-out) like NTFS-3G. Supports **extents** (contiguous block ranges) reducing metadata overhead for large sequential video files vs NTFS clusters.

2. Advanced Features for Data Integrity/Scale: **Metadata Checksums** (detect silent corruption), **Delayed Allocation** (improves write performance, reduces fragmentation), **Online Defragmentation (e4defrag)**, **Large Inode Support** (stores extended attributes/xattrs for ACLs/SELinux efficiently), **TRIM/Discard Support** (essential for SSD RAID).

(4) NTFS on SD Card / Camera Failure:

1. SD Association Standard: SDXC cards (64 GB – 2 TB) are mandatorily formatted exFAT by specification. Cameras firmware expects exFAT directory structure and cluster alignment. NTFS is not part of the SD spec.

2. Write Overhead / Wear: NTFS is a journaling file system. It writes to the `$LogFile` and updates `$MFT` on every metadata change (file create, timestamp update). This causes excessive write amplification on flash memory (no wear levelling controller as sophisticated as SSD), killing the card prematurely.
3. Cluster Size Mismatch: Windows default NTFS cluster = 4 KB. exFAT default for 64 GB = 128 KB. Camera firmware optimises write buffers for large clusters. Small NTFS clusters cause massive FAT/MFT updates per photo/video frame.

Exam Tip: "Format USB for Windows + Mac + Linux + Files > 4 GB" → exFAT. "Windows System Drive" → NTFS. "Linux System Drive" → ext4 (or Btrfs/ZFS). "UEFI Boot Partition" → FAT32.

METHOD **Integration: Troubleshooting Boot and Storage Scenarios**

1. Analyse the Symptom: "No boot device", "GRUB rescue>", "BitLocker recovery", "Raw file system", "Disk unknown/not initialized".
2. Identify the Layer: Firmware (UEFI/BIOS) → Partition Table (GPT/MBR) → Bootloader (ESP/MBR code) → OS Loader (Windows Boot Manager / GRUB / kernel) → Kernel → Init/Userland.
3. Apply Standard Fixes:
   - UEFI + GPT + "No Boot": Check Boot Order, Secure Boot state, ESP exists (FAT32), `.efi` files present. Repair: `bcdboot` (Win) / `grub-install` (Linux) from Live USB.
   - MBR + "GRUB rescue": `set prefix=(hd0,msdos1)/boot/grub, insmod normal, normal`. Then `grub-install /dev/sda`.
   - BitLocker Recovery: Requires Recovery Key (Azure AD, Microsoft Account, Printout, USB). Without key → Data loss (by design).
   - Raw File System: `chkdsk /f /r` (NTFS), `fsck -y` (ext4). Check SMART data first (`smartctl -a`).
   - Disk "Not Initialized": Initialize → Choose GPT (modern) → New Simple Volume → Format. Data lost if not recovered first.
4. Document: Always note error codes (0×c000000e, 0×c0000225), LED patterns, beep codes.

WORKED EXAMPLE 16 **Integrated Troubleshooting: Boot Failure and Disk Errors**

Statement: A teacher's laptop (Windows 11, UEFI, GPT, BitLocker on C:) displays a blue

screen: `INACCESSIBLE_BOOT_DEVICE` (Stop Code 0×7B) after a forced Windows Update restart. BitLocker recovery screen appears asking for the 48-digit key. The teacher has no key backup.

1. Explain the probable cause of the 0×7B error in the context of a Windows Update on a BitLocker-encrypted drive.
2. The teacher finds the BitLocker Recovery Key in their Microsoft Account online. After entering it, Windows enters Automatic Repair → "Startup Repair couldn't repair your PC". Log file shows `BadSystemFileInfo`. List the three command-line repair steps (from WinRE Command Prompt) to fix the boot files without data loss.
3. If Startup Repair fails completely, the teacher boots a Linux Live USB (Ubuntu) to backup files before reinstalling. The BitLocker partition is locked. Name the Linux tool to unlock it and the command to mount it read-only for backup.
4. After backup, the teacher does a clean install. The new Windows Setup sees the SSD as "Unallocated Space" but the ESP (100 MB) and Recovery Partitions remain from the old install. Should they delete all partitions or keep the ESP? Justify.

SOLUTION

Detailed Solution:
(1) Probable Cause of 0×7B (INACCESSIBLE_BOOT_DEVICE) with BitLocker: Windows Update modified the Boot Configuration Data (BCD) or critical boot drivers (`bootmgfw.efi`, `winload.efi`, storage drivers) on the encrypted volume. During early boot, the Windows Boot Manager (on ESP, unencrypted) hands off to `winload.efi` (on C:, encrypted). If the BitLocker driver (`fvevol.sys`) or the updated storage driver fails to load/decrypt the volume because the BCD signature/hash mismatch (Secure Boot policy update) or driver version mismatch, the OS cannot access the boot partition → 0×7B. The BitLocker Recovery Screen appears because the TPM measurements (PCR 7, 11) changed due to the update, triggering "BitLocker Recovery Mode".
(2) Three Command-Line Repair Steps (WinRE → Troubleshoot → Advanced → Command Prompt):

1. Identify Volumes: `diskpart` → `list vol`. Note drive letters: ESP (FAT32, no letter, usually 100–500 MB), Windows (NTFS, usually `C:` or `D:` in WinRE). Exit `diskpart`.
2. Repair Boot Files (BCD + Bootloader): `bcdboot C:\Windows /s S: /f UEFI` (Assumes C: is Windows, S: is ESP. Assign letter to ESP first: *`sel vol <esp_num>`*, *`assign letter=S`*). This recreates BCD and copies latest `bootmgfw.efi`, `winload.efi` from `C:\Windows` to ESP.
3. Check File System Integrity: `chkdsk C: /f /r` Fixes file system errors (`BadSystemFileInfo`) and recovers bad sectors.

Reboot.
(3) Linux Tool for BitLocker: **Dislocker** (or `cryptsetup` with `bitlocker` module kernel 5.7+).

Preferred Modern Method (Kernel 5.7+):

```
# 1. Unlock (creates /dev/mapper/bitlocker)
sudo cryptsetup bitlockerOpen /dev/nvme0n1p3 bitlocker
# Enter recovery password (48 digits) or .bek file: --key-file recovery.bek

# 2. Mount read-only (NTFS)
sudo mount -o ro /dev/mapper/bitlocker /mnt/backup
# Backup files from /mnt/backup to external drive
```

Legacy Dislocker Method:

```
sudo dislocker -r -V /dev/nvme0n1p3 -uPASSWORD -- /media/bitlocker
sudo mount -o ro,loop /media/bitlocker/dislocker-file /mnt/backup
```

(4) Clean Install Partition Strategy: **Delete ALL partitions** (including old ESP and Recovery). Justification:

1. Clean State: Old ESP may contain stale/corrupted boot entries, old BitLocker metadata, or incorrect Secure Boot keys. Windows Setup creates a fresh, correctly sized ESP (100 MB default, or 500 MB for 4K native drives) and Recovery partition matched to the new OS version.
2. Size Alignment: Old ESP might be 100 MB (512e drive). New drive/OS might prefer 500 MB. Old Recovery partition size may not match new WinRE image size.
3. No Dual Boot: Since it's a single OS clean install, keeping old partitions wastes space and risks "System Reserved" confusion (drive letters shifting).
4. Procedure: In Setup "Where do you want to install?", select each partition → Delete. Select "Unallocated Space" → Next. Windows auto-creates: Recovery, ESP, MSR, C:.

Critical Warning: Deleting partitions destroys data. Ensure backup (Step 3) is verified complete before Step 4.

8 EXERCISES

8.1 I check my knowledge

EXERCISE 8 **MCQ: System software identification**

For each of the following, choose the correct category (System Software or Application Software) and justify your choice in one sentence.

1. Windows 11
2. Microsoft Excel
3. A printer driver
4. An antivirus program
5. A C++ compiler
6. WhatsApp

EXERCISE 9 **True/False with justification**

State whether each statement is True or False. If False, correct it.

1. The kernel is the part of the operating system that interacts directly with the user.
2. A batch processing operating system allows multiple users to interact with the computer simultaneously.
3. The command-line interface (CLI) requires more memory resources than a graphical user interface (GUI).
4. Formatting a partition with NTFS makes it readable and writable by both Windows and Linux without additional software.
5. A compiler translates the entire source code into machine code before execution, while an interpreter translates and executes line by line.

EXERCISE 10 **Definitions**

Define the following terms clearly and concisely:

1. System software

2. Utility software
3. Device driver
4. File system
5. Partition
6. Bootloader

EXERCISE 11 **Direct questions**

1. List four main functions of an operating system.
2. Name two examples of single-user, single-tasking operating systems.
3. What is the difference between a cold boot and a warm boot?
4. Give the default file system used by modern versions of Windows and by most Linux distributions.

8.2 I practise

EXERCISE 12 **Operating system functions in a cybercafé**

State four functions of an operating system and explain each with a concrete example from daily computer use in a cybercafé in Yaoundé.

EXERCISE 13 **Comparing OS types**

Compare batch processing and time-sharing operating systems under the following headings:

1. User interaction
2. CPU utilisation
3. Typical use case (give one situation where each is appropriate)

EXERCISE 14 **Disk partitioning and installation**

A student wants to install Windows 11 on a 500 GB SSD. The student also wants to keep a separate partition for personal data and possibly install Linux later.

1. Propose a partitioning plan (sizes, file systems, labels) and justify your choices.
2. List the main steps of a clean Windows installation in the correct order, from booting the installation media to the first login.

EXERCISE 15 **CLI commands: Windows vs Linux**

Given the directory tree: `C:\Users\Amina\Documents\GCE\notes.docx` (Windows) and `/home/amina/Documents/GCE/notes.docx` (Linux).

1. Write the absolute path of `notes.docx` in each system.
2. Write the CLI commands to create a folder named `Revision` inside `GCE` and then copy `notes.docx` into it, for both Windows (Command Prompt) and Linux (Bash).

8.3 I prepare for the GCE

EXERCISE 16 **Compiler vs Interpreter** **(6 marks)**

1. Explain the difference between a compiler and an interpreter. (3 marks)
2. Name one programming language typically associated with each. (2 marks)
3. State one advantage of using an interpreter during program development. (1 mark)

EXERCISE 17 **Troubleshooting: "Operating system not found"** **(8 marks)**

A computer displays the message `Operating system not found` at startup.

1. Describe three possible causes of this error. (3 marks)
2. Outline the steps you would take to diagnose and repair the machine, in order. (5 marks)

EXERCISE 18 **CLI vs GUI** **(6 marks)**

1. Give two advantages of the command-line interface (CLI) over the graphical user interface (GUI). (2 marks)
2. Give two disadvantages of the CLI compared to the GUI. (2 marks)
3. A network administrator needs to create 50 user accounts with specific permissions on a Linux server. Which interface would be more efficient and why? (2 marks)

9 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ: System software identification**

1. Windows 11 — System Software. It is an operating system: it manages the hardware, memory, files and all other programs running on the computer.
2. Microsoft Excel — Application Software. It performs a specific task for the end user (spreadsheets and calculations), not the management of the machine.
3. A printer driver — System Software. It enables the operating system to communicate with and control a hardware device (the printer).
4. An antivirus program — System Software (utility software). It maintains and protects the system by scanning for and removing malware; it serves the computer rather than a user's production task.
5. A C++ compiler — System Software (language processor). It translates source code into machine code, a service to the functioning of the system and to programmers, not an end-user task.
6. WhatsApp — Application Software. It performs a user-oriented task (messaging and communication).

SOLUTION **Exercise 2 — True/False with justification**

1. False. The **kernel** is the core of the operating system that interacts directly with the hardware (CPU, memory, devices). It is the shell or user interface that interacts with the user.
2. False. A batch processing system executes jobs one after another without user interaction. It is a multi-user / time-sharing operating system that allows several users to interact with the computer simultaneously.
3. False. The CLI requires fewer memory and processing resources than a GUI, because it does not need to manage windows, icons, menus and graphical rendering.
4. False. NTFS is native to Windows. Linux can read NTFS (and usually write it with the NTFS-3G driver), but Windows cannot natively read Linux file systems such as ext4. For a partition readable and writable by both systems without extra software, one would use FAT32 or exFAT.
5. True. A compiler translates the whole source program into machine code (an exe-

cutable) before it is run, whereas an interpreter translates and executes the code line by line at run time.

SOLUTION **Exercise 3 — Definitions**

1. System software: the set of programs that manage, control and support the computer hardware and provide a platform on which application software runs (e.g. operating systems, drivers, utilities, language processors).
2. Utility software: system software designed to analyse, configure, optimise, protect or maintain the computer (e.g. antivirus, disk defragmenter, backup tools, compression tools).
3. Device driver: a small program that allows the operating system to communicate with and control a specific hardware device (printer, keyboard, graphics card…).
4. File system: the method and data structures an operating system uses to organise, name, store, retrieve and keep track of files on a storage device (e.g. NTFS, FAT32, ext4).
5. Partition: a logically independent section of a physical storage disk, treated by the operating system as a separate drive, which can hold its own file system.
6. Bootloader: a small program loaded by the firmware (BIOS/UEFI) at startup, whose role is to locate and load the operating system kernel into main memory (e.g. GRUB, Windows Boot Manager).

SOLUTION **Exercise 4 — Direct questions**

1. Four main functions of an operating system:
   - Process (task) management — creating, scheduling and terminating processes.
   - Memory management — allocating and freeing main memory for programs.
   - File management — organising, storing and retrieving files and folders.
   - Device (input/output) management — controlling peripherals through drivers. (Also acceptable: security/user management, providing a user interface.)
2. Examples of single-user, single-tasking operating systems: MS-DOS and Palm OS (early versions).
3. A cold boot is starting the computer from a completely powered-off state (hardware is initialised, the full POST runs). A warm boot is restarting the computer while it is al-

ready on (e.g. choosing “Restart”), without switching the power off; some hardware initialisation is skipped.

4. Modern Windows: NTFS. Most Linux distributions: ext4.

SOLUTION **Exercise 5 — Operating system functions in a cybercafé**

1. Process management: the OS schedules the CPU among the many programs a customer runs at once — for example a customer in a Yaoundé cybercafé browsing with Chrome, typing a letter in Word and listening to music simultaneously, without the machine freezing.
2. Memory management: the OS allocates RAM to each open application and reclaims it when the program closes, so that a customer who closes a video frees memory for the next customer’s work.
3. File management: the OS organises each customer’s documents in folders on the disk, controls saving to USB flash drives, and allows the manager to delete a customer’s files after the session.
4. Device management: the OS uses drivers so that any customer can print a document on the shared printer, plug in a flash drive, or use the keyboard and mouse without configuring the hardware themselves.

(Also acceptable: security and user management — the manager logs in with an administrator account while customers use a limited account, protecting the system settings.)

SOLUTION **Exercise 6 — Comparing OS types**

Criterion	Batch processing	Time-sharing
User interaction	None during execution: jobs are collected, queued and run one after another without the user intervening.	Direct and simultaneous: many users interact with the system at the same time through terminals.
CPU utilisation	CPU processes jobs sequentially; it may idle while a job waits for input/output, but throughput for large repetitive jobs is high.	CPU time is divided into small slices (quantum) shared among users, keeping the CPU busy and giving each user a fast response.
Typical use case	Processing the monthly payroll or bank statements of a company overnight, where identical treatment is applied to thousands of records.	A university or bank server where many students/customers connect simultaneously and each needs an immediate response.

SOLUTION **Exercise 7 — Disk partitioning and installation**

1. Proposed partitioning plan (500 GB SSD):

Partition	Size	File system	Justification
EFI System Partition	≈ 0.5 GB	FAT32	Required by UEFI to store bootloaders (created automatically by the installer).
C: (Windows)	150–200 GB	NTFS	Enough space for Windows 11, updates and installed applications; NTFS is the native Windows file system (security, large files).
D: (Data)	200–250 GB	NTFS (or exFAT)	Personal data kept separate from the OS: Windows can be reinstalled without losing documents; exFAT would also be readable by Linux.
Unallocated space	≈ 100 GB	(none yet)	Reserved for a future Linux installation; the Linux installer will create its ext4 and swap partitions there.

Keeping data on a separate partition protects it during OS reinstallation, and leaving unallocated space avoids shrinking partitions later.

2. Main steps of a clean Windows installation:

1. Create a bootable Windows 11 USB medium and boot the computer from it (selecting it in the BIOS/UEFI boot menu).
2. Choose language, time and keyboard settings, then click “Install now”.
3. Enter the product key (or choose “I don’t have a product key”) and select the Windows edition.
4. Accept the licence terms and choose Custom: Install Windows only (advanced).
5. Create/select the partition for Windows (delete old partitions if a truly clean install is wanted) and click Next — Windows copies and installs the files; the machine restarts several times.
6. Complete the out-of-box experience: region, keyboard layout, network connection, user account (Microsoft or local account) and privacy settings.
7. Reach the desktop, install drivers and updates, then log in for the first normal session.

SOLUTION **Exercise 8 — CLI commands: Windows vs Linux**

1. Absolute paths:

- Windows: `C:\Users\Amina\Documents\GCE\notes.docx`
- Linux: `/home/amina/Documents/GCE/notes.docx`

Note the differences: Windows uses a drive letter (`C:`) and backslashes `\`; Linux starts from the root `/` and uses forward slashes `/`.

2. Commands:

- Windows (Command Prompt):
  - `cd C:\Users\Amina\Documents\GCE`
  - `mkdir Revision`
  - `copy notes.docx Revision\`
- Linux (Bash):
  - `cd /home/amina/Documents/GCE`
  - `mkdir Revision`
  - `cp notes.docx Revision/`

WARNING **Case sensitivity**

Linux file names and commands are case-sensitive: `Revision` and `revision` are different folders, and `MKDIR` would fail. Windows is generally not case-sensitive.

SOLUTION **Exercise 9 — Compiler vs Interpreter (6 marks)**

(a) Difference (3 marks): A compiler translates the entire source program into machine code in one pass, producing an executable file which can then be run many times without the source code; errors are reported after the whole program has been analysed. An interpreter translates and executes the source code line by line at run time, producing no separate executable; execution stops at the first error encountered. Consequently compiled programs generally run faster, while interpreted programs are translated afresh at every execution.
(b) Examples (2 marks): Compiler: C (or C++, Pascal). Interpreter: Python (or BASIC, JavaScript).
(c) Advantage of an interpreter during development (1 mark): errors are detected and reported immediately, line by line, so the programmer can test and correct small portions of code quickly without recompiling the whole program — this speeds up debugging and learning.
Examiner expectations / mark scheme: 1 mark for "whole program vs line by line", 1 mark for "executable produced vs none", 1 mark for a consequence (speed or error handling); 1 mark per correct language; 1 mark for a valid development advantage. Any technically correct equivalent earns the marks.

SOLUTION **Exercise 10 — Troubleshooting: "Operating system not found" (8 marks)**

(a) Three possible causes (3 marks — 1 each, any three):

- The hard disk/SSD containing the OS is not detected (loose or faulty data/power cable, failed disk).
- The boot order in the BIOS/UEFI is wrong (the machine tries to boot from an empty USB drive, DVD or network).
- The bootloader or boot files are corrupted (e.g. damaged MBR/EFI partition after a power cut or failed update).
- The system partition has been deleted or corrupted, or the disk has physically failed.

(b) Diagnosis and repair steps, in order (5 marks):

1. Remove external media (USB drives, DVDs) and restart — the machine may simply be trying to boot from them.
2. Enter the BIOS/UEFI setup and check whether the hard disk/SSD is detected and whether it is first in the boot order; correct the boot order if necessary.
3. Check the hardware connections (open the case if a desktop, reseat the disk's data and power cables) or test the disk in another machine / with another cable.
4. Boot from a Windows installation or recovery USB and use the repair tools (Startup Repair, or rebuild the boot configuration with commands such as `bootrec /fixmbr` and `bootrec /rebuildbcd`).
5. If the disk is detected but repair fails, back up data (e.g. via a live Linux USB) and reinstall the operating system; if the disk is not detected anywhere, replace the disk and reinstall.

Examiner expectations: marks are awarded for a logical order — simple external causes first, then firmware settings, then hardware, then software repair, and reinstall/replacement only as a last resort. Any coherent, technically sound sequence earns full credit.

SOLUTION **Exercise 11 — CLI vs GUI (6 marks)**

(a) Two advantages of the CLI (2 marks):

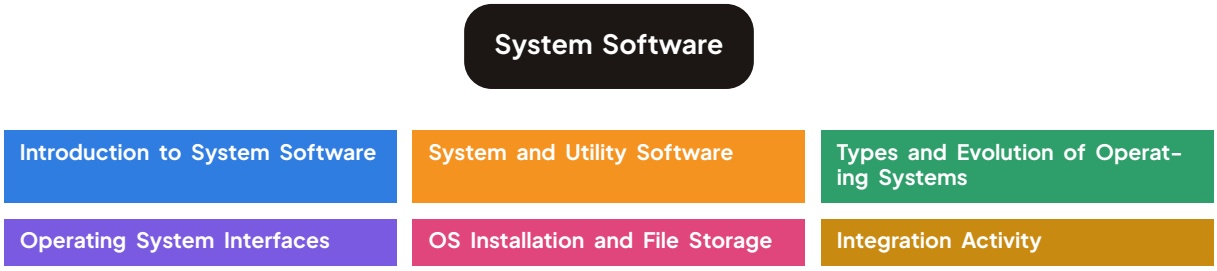
- It uses far fewer resources (memory, CPU) than a GUI — important on servers.
- Repetitive tasks can be automated with scripts and commands can be combined, making bulk operations very fast. (Also acceptable: faster for expert users; precise control; usable remotely over low-bandwidth connections.)

(b) Two disadvantages of the CLI (2 marks):

- Commands and their syntax must be memorised; a single typing error can cause failure or damage.
- It is less intuitive / harder to learn for beginners, with no visual cues such as icons and menus.

(c) Creating 50 user accounts (2 marks): the CLI is more efficient. The administrator can write a short script (e.g. a Bash loop reading a list of names and calling `useradd` with the right options) that creates all 50 accounts with their permissions automatically in seconds, whereas clicking through a GUI 50 times would be slow and error-prone. (1 mark for the choice, 1 mark for the justification based on automation/repetition.)
Examiner expectations: answers must be comparative (not just a property of the CLI); for part (c) the justification must refer to the repetitive, scriptable nature of the task.

10 SUMMARY SHEET



Mind map of Chapter CSC09: Exploring system software concepts

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key Definitions

- **Software**: The set of programs and data that instruct the hardware what to do; divided into system software and application software.
- **System Software**: Programs that manage and control the hardware resources and provide a platform on which application software runs (e.g., operating systems, device drivers, firmware, translators such as compilers).
- **Application Software**: Programs that perform tasks for the end user (e.g., word processors, spreadsheets, the school fees management program used in many Cameroonian colleges).
- **Utility Software**: Tools that perform maintenance, optimisation, or security tasks to keep the system running well (e.g., disk defragmenter, antivirus, backup utility, disk cleanup).
- **Operating System (OS)**: The core system software that controls the hardware, manages processes, memory, files and input/output devices, and provides the user interface. Examples: Windows, Linux, macOS, Android.
- **Kernel**: The central part of the OS, resident in memory, that interacts directly with the hardware (process scheduling, memory management, device management).
- **Shell**: The user interface layer (command-line or graphical) that interprets user commands and passes requests to the kernel.
- **Device Driver**: A small program that allows the OS to communicate with a specific hardware device (printer, flash drive, network card).
- **Firmware**: Software permanently stored on a hardware component (e.g., the BIOS/UEFI stored on the motherboard ROM).

- **File System**: The logical structure used to organise, store, name and retrieve files on a storage medium (e.g., NTFS, FAT32, ext4).
- **Boot Loader**: A small program that loads the OS kernel into main memory during start-up (e.g., GRUB for Linux, Windows Boot Manager).
- **Process**: A program in execution, with its own memory space and resources.

Laws / Principles / Models (Examinable)

- Layered Architecture Principle: System software is organised in layers: hardware → kernel → system utilities and libraries → shell → application software. Each layer uses the services of the layer immediately below it and offers services to the layer above.
- Kernel Mode vs User Mode: The kernel executes in kernel (privileged) mode with full access to the hardware; applications and most utilities execute in user mode and must request services from the kernel. This protects the system from faulty or malicious programs.
- Process State Model: A process moves among the states New, Ready, Running, Waiting (Blocked) and Terminated. The scheduler selects which ready process runs next.
- Memory Management Roles: The OS allocates memory to processes, and may use paging, segmentation and virtual memory to let programs run even when larger than the available RAM.
- File Allocation Methods: Contiguous, linked and indexed allocation; know the advantages and disadvantages of each regarding fragmentation and speed of access.
- OS Evolution Stages: Batch processing → multiprogramming → time-sharing (multitasking, multi-user) → real-time → network/distributed → mobile and embedded systems. Be able to state the key characteristic of each stage.

Methods / Step-by-Step Procedures

- OS Installation (typical steps):
  1. Check hardware requirements and back up important data.
  2. Prepare bootable media (USB/DVD) containing the OS image.
  3. Configure the BIOS/UEFI boot order to start from the installation media.
  4. Partition the disk: choose sizes and file systems for the OS partition, swap area (Linux) and data partition.
  5. Run the installer: select language, time zone, keyboard layout; create user accounts and passwords.
  6. Install the boot loader (usually done automatically by the installer).
  7. Restart, apply updates, install device drivers, and configure network and security settings.

- Creating and Using a File System (example: ext4 on Linux):
  1. Identify the target partition (e.g., `/dev/sdb1`).
  2. Format it: `mkfs.ext4 /dev/sdb1`.
  3. Mount it: `mount /dev/sdb1 /mnt/data`.
  4. Add an entry to `/etc/fstab` so it mounts automatically at start-up.
- Using a Command-Line Interface (CLI) for System Tasks:
  1. Open a terminal (Linux) or command prompt (Windows).
  2. Navigate with `cd`; list files with `ls` (Linux) or `dir` (Windows).
  3. Run administrative commands with elevated privileges: `sudo` on Linux, Run as administrator on Windows.
  4. Use redirection (>, >>) and pipes (|) to combine commands and automate tasks.

Common Mistakes (Traps)

- Confusing system software with application software. System software manages the machine (OS, drivers, utilities); application software serves the user's own tasks (word processor, browser).
- Believing that all system software runs in kernel mode. Only the kernel and drivers run in kernel mode; utilities such as disk cleanup run in user mode like ordinary programs.
- Treating a utility as part of the kernel: a utility is system software, but it is a separate tool, not a component of the kernel.
- Mixing up program and process: a program is a passive file on disk; a process is that program in execution, with memory and resources allocated by the OS.
- Assuming all operating systems use the same file system. Know at least three: NTFS (modern Windows), FAT32 (older Windows, flash drives), ext4 (Linux).
- In installation questions, forgetting to set the correct boot device in the BIOS/UEFI, or omitting the partitioning and boot-loader steps.
- Misclassifying operating systems: e.g., Windows 95 is a single-user, multi-tasking GUI operating system, not a real-time OS; a real-time OS (e.g., in an air-traffic or medical monitoring system) must respond within strict time limits.

GCE Examination Tips

- Definitions: Give concise, textbook-style definitions and add one example (e.g., "Kernel – the core of the OS that manages hardware directly, e.g., the Linux kernel").
- Diagrams: Be ready to draw and fully label (i) the layered architecture of a computer system and (ii) the process state transition diagram with all arrows named.

- Comparison Tables: Prepare a table comparing at least three OS types (batch, time-sharing, real-time) using criteria such as user interaction, response time and typical applications.
- Command Syntax: Know basic CLI commands for both Windows (`dir`, `copy`, `tasklist`) and Linux (`ls`, `cp`, `ps`); examiners may ask for the exact command.
- File System Questions: Be able to explain fragmentation and to justify a file system choice for a scenario (e.g., "NTFS for a Windows system partition because it supports file permissions and very large files").
- Integration Activity: Expect a scenario asking you to select an OS, a partition scheme, a file system and utilities for an organisation (e.g., equipping a school computer laboratory in Yaoundé). Structure your answer: requirements → choices → justification.
- Time Management: Spend about 10 minutes on definitions, 15 minutes on diagrams and tables, and 20 minutes on a long-answer scenario. Use headings to keep answers structured.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+